

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA DELLE SCIENZE DI BASE
DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

Control Methods for a Dual-Arm Robotic Manipulation System

**CORSO DI LAUREA MAGISTRALE IN INGEGNERIA DELL'AUTOMAZIONE
E ROBOTICA**

Relatori:

Prof. Matjaz Mihelj
Prof. Mario Selvaggio

Candidato:

Salvatore De Luca
P38000281

Anno Accademico 2025/2026

A nonno

Possiamo passare la vita a farci dire dal mondo cosa siamo. [...]
A lasciare che la storia ci spieghi se siamo buoni o cattivi.
A lasciare che sia il passato a decidere il nostro futuro.
Oppure possiamo scegliere da noi.
E forse inventare qualcosa di meglio è proprio il nostro compito. [...]
Qui, in mezzo alle rovine e al buio,
quello che stiamo costruendo potrebbe essere qualsiasi cosa.

Chuck Palahniuk, *Soffocare* (2001)

Acknowledgments

Alla fine di un percorso di studi di sei anni è normale fare un bilancio e in questi ultimi mesi è stato inevitabile per me. Non sono una persona lineare o così decisa sulle scelte della vita, e, sebbene questo dipenda anche dall'insicurezza, ho capito che è una naturale conseguenza della giovinezza o semplicemente di chi sono. Una volta, durante l'ultimo anno di liceo ci chiesero cosa volessimo fare nella vita e l'unica cosa che mi venne in mente da dire fu: "Vorrei riuscire bene in ciò che farò". In quel momento era una frase senza importanza e magari dettata più dal non sapere cos'altro dire, ma, adesso, benché siano cambiate tantissime cose, guardo con ammirazione, e con un po' di nostalgia, quel ragazzo così ignaro di tutto e di cosa avrebbe fatto, eppure con il coraggio di buttarsi, nonostante tutto. Spero con tutto me stesso di essere riuscito almeno in parte a renderti giustizia. Questo percorso sicuramente non sarebbe stato lo stesso senza chi ha scelto di starmi accanto e aiutarmi, più o meno direttamente. L'elenco che segue non è in alcun modo stilato in ordine di importanza e vi ringrazio tutti dal profondo del cuore.

Alla mia famiglia (e a Pupetta), so di non essere stato il figlio e il fratello più semplice da avere accanto in questi anni e vi ringrazio per la pazienza e la fiducia che mi avete dato. Vi voglio tanto bene, anche se a volte non riesco a dirlo.

A tutti i miei compagni di corso, grazie di tutti i momenti passati insieme e del supporto reciproco. In particolare grazie a Ciro, Gennaro, Mery, Alessia, Pietro e Gabriele. Non è stato un percorso sempre semplice per me, ma la vostra presenza mi ha aiutato davvero a vivere in modo un po' più sano questi anni. Anche se prenderemo strade diverse, le esperienze, le risate ed i ricordi insieme rimarranno sempre con me e per questo vi sono davvero grato.

A Giuseppe, sono profondamente grato che tu faccia parte della mia vita e questi ultimi mesi non sarebbero stati gli stessi senza di te. Guardandomi indietro sento di essere una persona migliore, e questo un po' lo devo anche a te. So che ti imbarazza se lo dico, ma ti voglio bene (in un senso *squisitamente* partenopeo).

A Irene, faccio difficoltà a dire quanto sia stato significativo averti accanto in questi anni e ricevere il tuo supporto. Vorrei scrivere qualcosa di lungo, che sia all'altezza di ciò che c'è stato, ma in questo sei sempre stata più brava tu. Penso a qualche anno fa, quando stavamo seduti sotto casa tua a parlare di qualunque cosa e a condividere il più possibile, e fa strano pensare a tutta la strada che abbiamo fatto. Più gli anni passano e più mi accorgo che per me le cose più importanti sono i ricordi: grazie per avermene dati così tanti. Ti voglio bene.

A Ilaria, ci siamo rincontrati da relativamente poco tempo, ma mi sembra che non sia passato così tanto ed è davvero bello e consolante scoprire come certe cose rimangano le stesse, nonostante la vita che cambia. Ti voglio bene e sono felice che ci siamo ritrovati.

A Maria, ti sono molto grato per esserci stata. Anche se abbiamo preso percorsi diversi, sono felice di essere cresciuto anche un po' insieme a te. Ti voglio bene.

Un ringraziamento speciale va a Silvia, Emanuele, Daniele e Giulia: stiamo facendo tutti esperienze e vite diverse, ma il ricordo e il sostegno che ci siamo dati sono stati preziosi e rimarranno sempre con me. Grazie di tutto.

Abstract

This thesis proposes the development of control architectures for a dual-arm robotic system operating in a shared workspace. The use of a dual-arm architecture confers greater dexterity and design freedom compared to single-manipulator counterparts, especially in visual servoing tasks. As a primary contribution, the project involved creating the complete ROS 2 software interface from scratch for two manipulators recently acquired by the RoboLab (University of Ljubljana). Building upon this setup, the main objective is to guarantee the simultaneous control and collaborative execution of tasks, solving the critical problem of collision avoidance. For this purpose, two distinct methodologies have been implemented and validated on the Neura LARA8 platform: a kinematic planning approach based on the MoveIt framework and a reactive architecture developed natively in `ros2_control`. While the first approach resolves interferences at the global planning level, the second exploits a visual servoing law combined with the projected gradient method, translating into internal motions of the manipulator that ensure safe collision avoidance. Furthermore, an additional methodological contribution addresses the problem of occlusion avoidance by formulating a constrained optimization problem, which has been solved using a Quadratic Programming (QP) solver integrated within the `ros2_control` framework; this occlusion-avoidance strategy was further validated on a separate dual-arm setup. Experimental results confirm the robustness of the reactive architecture: in the presence of spatial disturbances introduced by the second manipulator, the system guarantees strict collision and occlusion avoidance while maintaining a zero steady-state error on the visual features of the primary task.

Keywords: Dual-arm, Visual servoing, Collision avoidance, Occlusion avoidance, Quadratic programming, ROS 2.

Contents

Acknowledgments	iii
Abstract	v
Contents	vi
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	2
1.2 State of the Art	5
1.2.1 Applications of Dual-Arm Systems	5
1.2.2 Control Strategies for Dual-Arm Systems	6
1.2.3 Occlusion Avoidance in Visual Servoing	7
1.2.4 Frameworks and Implementation Solutions	8
1.3 Structure of the thesis	9
2 Methodology	11
2.1 Differential Kinematics	12
2.2 Image Based Visual Servoing	14
2.3 Projected Gradient Method	15
2.4 Occlusion-free Visual Servoing	18
3 Experiments and Results	22
3.1 Experimental setup	22
3.1.1 Driver modification	25
3.2 MoveIt Integration	26
3.2.1 Validation of the MoveIt Motion Planner	27
3.3 Preliminary Validation via Vision-Guided Grasping	28
3.3.1 YOLOv8 Model	30
3.3.2 Grasping Sequence	31
3.4 Advantages and Limitations of MoveIt	32
3.5 Custom Control Architecture	34
3.5.1 Visual Servoing	34

3.5.2	Visual Servoing Gradient Projected Method	37
3.5.3	Occlusion-Free Visual Servoing	43
4	Conclusions	54
	Bibliography	56
	Web References	59

List of Figures

1.1	Neura LARA 8 manipulators.	4
2.1	IBVS Control Scheme.	15
2.2	PGM Visual Servoing Control Scheme.	17
2.3	Occlusion-free Visual Servoing Scheme.	19
3.1	Camera (a) and gripper (b) mounted on the two manipulators.	23
3.2	Setup of the two manipulators.	23
3.3	Gazebo environment (a) and simulated camera image (b).	24
3.4	Experimental setup of the dual Franka Emika Panda robotic system.	25
3.5	Neura LARA 8 Manipulators in RViz.	27
3.6	Planning Scene in RViz.	28
3.7	Example of usage of the MoveIt plugin in RViz GUI.	29
3.8	Yolo Model detection results.	30
3.9	Nodes and topics architecture of the detection task	31
3.10	Object's frame.	32
3.11	Grasping pipeline using MoveIt.	33
3.12	Setup for the Visual Servoing task.	36
3.13	Centroid Coordinates (IBVS).	37
3.14	Commanded joint velocities (IBVS).	38
3.15	Collisions volumes before (a) and after the change (b).	39
3.16	Experiment phases.	42
3.17	Centroid Coordinates (PGM).	42
3.18	Cost evolution.	43
3.19	Commanded right joints velocities (PGM).	44
3.20	Commanded left joints velocities (PGM).	45
3.21	Experiment phases.	46
3.22	Centroid Coordinates (Grasping task).	46
3.23	Cost function (Grasping task).	47
3.24	Detailed view of the left manipulator with the occluding object.	49

3.25	Nodes and topics architecture for the Occlusion-Free IBVS.	49
3.26	Occlusion-free visual servoing results.	50
3.27	Image Space Error Occlusion-free VS.	51
3.28	Commanded Twist (Camera Arm).	52
3.29	Commanded Twist (Right Arm).	53

List of Tables

3.1	IBVS Results.	35
3.2	PGM Visual Servoing parameters.	40
3.3	Analysis of the QP solver execution time per cycle.	51
3.4	PGM Visual Servoing parameters in the Occlusion-free VS Case.	52

Chapter 1

Introduction

Dual-arm robotic manipulation has progressively emerged as a natural evolution of robotic systems, moving beyond the limitations of single-arm architectures toward configurations that more closely resemble human dexterity. This shift is motivated by the growing demand for robots capable of operating in unstructured and dynamic environments, where the kinematic redundancy and cooperative capabilities offered by two coordinated manipulators enable tasks that a single arm could not accomplish, or could accomplish less efficiently. However, exploiting this potential introduces a set of challenges. Coordinating two manipulators operating in a shared workspace requires solving the problem of collision avoidance between each other, while simultaneously deciding how responsibilities should be distributed between the two arms. The literature has proposed several strategies to address these problems, from task-priority and complementary motion control schemes to methods based on Hierarchical Quadratic Programming (HQP). This thesis addresses these challenges by developing and validating a control architecture for a dual-arm system in which one manipulator, equipped with an eye-in-hand camera, performs Image-Based Visual Servoing while relying on the Projected Gradient Method to guarantee collision avoidance and the second manipulator carries out the physical interaction with the environment. A further contribution of this thesis work concerns the resolution of the occlusion avoidance problem in the context of visual servoing, addressed by formulating a constrained optimization problem based on Quadratic Programming (QP). Beyond the theoretical formulation, the work contributes an open-source ROS 2 [1] software infrastructure, developed from scratch

to overcome the limitations of the manufacturers' proprietary interfaces. This architecture was initially validated on two Neura LARA 8 collaborative manipulators at the Laboratory of Robotics (RoboLab) of the University of Ljubljana (Figure 1.1)¹. Furthermore, the latest developments of the thesis were deployed and tested on a different experimental setup featuring two Franka Emika Panda robots.

The chapter is organized as follows: first, Section 1.1 presents the core motivation of this thesis, defining the engineering problems under consideration and justifying the choices made through the thesis project working on a dual arm system. Section 1.2 reviews the current state of the art examining the areas of application and the challenges that arise when it comes to dual-arm architectures. Finally, Section 1.3 outlines the structural organization of the thesis, briefly illustrating the contents and focus of each subsequent chapter.

1.1 Motivation

This thesis project is focused on the setup of an architecture constituted by two collaborative robots, with the final aim of making the two robots perform a collaborative task with a visual servoing approach. The choice of using a dual-robot architecture instead of a single-robot architecture implies a series of issues to solve: on the one hand the most relevant is how to solve the problem of generating paths which are collision-free with respect to the other robot; on the other hand one should choose how to control the two robots, and two approaches can be considered in this regard: we can either consider the two-robot system as a monolithic system and in that case the two robots are conceived as one, or we can consider the two robots as two separate entities. The usage of two robots can be helpful with respect to a single-robot system. In visual servoing tasks one of the main issues is to ensure the camera of the robot does not lose the detected target(s) while the robot is moving, since if we lose visual information the controller will work in open-loop; this implies a strict control on the generated trajectory which should be easily predictable. In this regard, one simple

¹The ROS 2 workspace for the Neura LARA 8 setup is available at: <https://git.robolab.si/Salvatore/neuraLARA8>

solution could be to use a robot in charge of extracting visual information from the environment, and it will move so that the camera will never lose the detected target(s), whereas a second robot will be in charge of interacting with the environment and the target (for example picking it). This is not the only convenient architecture for exploiting a dual-arm system: for example one can think of using a fixed camera and both the robots can grasp/interact with the detected target(s), choosing in each instant which among the two robots is more convenient to use. The practical implementation and experimental validation of this research were carried out during a visiting period abroad at the Laboratory of Robotics (RoboLab), affiliated with the Faculty of Electrical Engineering at the University of Ljubljana. Specifically, the experimental activities were conducted on two Neura LARA 8 collaborative manipulators recently acquired by the laboratory (Figure 1.1). Operating on this hardware provided the opportunity to validate the proposed control architectures in a real-world scenario, addressing both the theoretical collision avoidance challenges and the low-level software integration required to synchronize the two robotic arms. Finally, one of the motivations behind this thesis project was the need to provide the host laboratory with an open-source software architecture based on ROS 2 (Robotic Operating System), which the lab did not have for the two newly acquired Neura LARA 8 manipulators. In fact, the proprietary C++ and Python APIs supplied by the manufacturer had such limitations that they did not allow synchronous communication between the two robots. Creating such an open-source infrastructure from scratch made it possible to provide a multi-robot system ready for advanced research and capable of performing collaborative tasks using complex algorithms. Moreover, the development of this infrastructure will serve as the foundation for an internal laboratory project focused on the application of robotics to agriculture. The software architecture validated in this thesis represents an important first step toward implementing future projects related to autonomous fruit harvesting (where occlusions from branches and leaves must also be handled), pruning, and the manipulation of agricultural products.



Figure 1.1 – Neura LARA 8 manipulators.

1.2 State of the Art

This section provides an overview of the literature and the technological solutions relevant to the development of this thesis. The section has been divided in four parts: in the first Section 1.2.1 are investigated the principal applicative fields of dual-arm robotic systems, highlighting challenges and difficulties and the principal techniques of control used to face these issues; Section 1.2.2 outlines the principal control strategies for bimanual systems; Section 1.2.3 presents solutions proposed in the literature for occlusion avoidance in visual servoing which have been considered significant for the purposes of this work; finally, in 1.2.4 are presented the principal frameworks used for the planning and the control of the above-mentioned systems.

1.2.1 Applications of Dual-Arm Systems

The use of dual-arm systems is gaining more interest given that they allow to solve complex tasks in consequence of their redundancy and of their human-like structure. Dual arm systems find applications in uncertain and dangerous environments (for the manipulation of delicate material) or also in domestic fields and, in particular agricultural. As described by Jo *et al.* [2], the agricultural sector results particularly challenging, since it requires the interaction with a complex and not structured environment. Moreover for actions of harvesting the dual-arm systems result particularly suitable both because they imitate the human figure and because they offer a great freedom in the choice of the setup: one can think to architectures in which to a robot is delegated the task of vision and to another the task of the grasping. Furthermore the problem of the harvesting is not of simple resolution also because of the type of manipulation to perform. An example of application is presented by Stavridis *et al.* [3], in which the architecture is thought in a way to delegate to a robot the vision and the cut of the stem, while to another the grasping in order to pull the fruit. While harvesting and pruning are prominent, dual-arm systems are also being explored for precise agricultural tasks. For instance, Rodríguez-Nieto *et al.* [4] introduce a dual-arm mobile robot for automated seeding in hydroponic greenhouses, demonstrating how visual perception and bimanual manipulation can be integrated to reliably deposit seeds into small

rock-wool cubes.

1.2.2 Control Strategies for Dual-Arm Systems

In the state of the art of controlling dual-arm robotic systems, strategies are divided into several categories to effectively manage cooperation, spatial interaction, and kinematic redundancy as classified by Jiang *et al.* [5]. As a significant approach in the literature, it is worth mentioning the *Complementary motion control*. This strategy coordinates the operations of the two arms by synchronizing their movements to improve efficiency and precision. For more complex tasks, the complementary approach breaks down the global task into simpler sub-tasks assigned to each arm (such as dividing workspaces to avoid spatial conflicts). An example of the application of advanced coordination techniques can be found in the work of Ahn *et al.* [6], where this strategy solves the problem of object retrieval in cluttered environments, where precedence constraints between tasks must be explicitly planned to maximize task-switching efficiency between manipulators. This method allows optimal workload balancing and low energy consumption, but has the limitation of requiring precise prior knowledge of the manipulated objects' characteristics and environment dynamics [5]. A practical application of this complementary approach is demonstrated by Rodríguez-Nieto *et al.* [4] for agricultural seeding tasks. In their dual-arm seeding strategy, both robotic arms are synchronized to simultaneously collect seeds and subsequently deposit them in parallel sub-tasks, significantly reducing the overall cycle time compared to a sequential single-arm operation. On the other hand, a widely used approach is *Task-priority control*, which improves system adaptability by hierarchically optimizing the execution order of operations to resolve conflicts and avoid collisions in complex spaces [5]. The use of the HQP as strategy of control is widely used as strategy in bimanual environment; an example is provided by Gursoy *et al.* [7], in which the problem of the self-collision between the two manipulators is solved using an HQP. Alternatives to the HQP method, but that anyway exploit the functional redundancy of bimanual systems, are for example the method of the projected gradient as described by Selvaggio *et al.* in [8]. In the present work of thesis has been used this last approach and the setup consists in a robot equipped

with camera to which is delegated the vision and another manipulator that occupies itself of the interaction with the environment. Following this task-priority philosophy, the projected gradient method has been utilized to establish a hierarchy for the tasks executed by the camera-equipped robot. This enables the manipulator to exhibit a reactive behavior with respect to the movements of the other arm.

1.2.3 Occlusion Avoidance in Visual Servoing

Managing visual occlusions is a critical challenge in unstructured environments. One traditional approach relies on artificial potential fields; for instance, Papageorgiou *et al.* [9] propose an eye-in-hand controller that uses potential fields based on point cloud data to unveil objects, although this method lacks the strict safety guarantees provided by formal optimization. A similar control approach is used by Stavridis *et al.* [3]. An alternative is the use of geometric constraints in a constrained optimization problem, as proposed by Nicolis *et al.* [10], where the occluding object is modeled via significant points. Further developments along this line of research are presented by Wang *et al.* [11], in which a more sophisticated strategy is adopted to create an occlusion-free visual servoing framework for robotic manipulators. In this case, the hierarchical control architecture envisages the use of a high-level Model Predictive Control (MPC) to accomplish the visual task (regulation of the target feature points), and, at a lower level, the use of a controller based on control barrier functions (CBF), integrated within a QP solver, which filters the nominal input while guaranteeing occlusion avoidance in continuous time. Moreover, this approach has the advantage of representing the objects (target and occluders) by means of the convex hull of their respective feature points, reducing conservativeness and improving numerical robustness and differentiability with respect to other approaches. This architecture guarantees rigorous occlusion avoidance in continuous time, overcoming the limitations of discrete-time MPC approaches, which may fail to maintain safety constraints in highly dynamic scenarios. A similar approach, but based on a different geometric formulation, is proposed by Wei *et al.* [12]: here, the convex polygonal projections of target and obstacles are embedded into strictly convex shapes by means of scaling functions, which smooth their corners

by replacing the linear constraints of the sides with a weighted sum in exponential form. A CBF is then constructed on these smooth shapes starting from the minimum scaling factor, i.e., the coefficient for which the two shapes, expanded while keeping their centers fixed, just touch each other: a value close to the safety threshold signals an imminent occlusion. Unlike [11], however, there is no MPC planning layer: the control is purely reactive (CBF-QP). In this thesis work, the occlusion avoidance problem has been addressed using a Quadratic Programming (QP) solver. Although this approach has proven to be effective for the considered scenarios, the exploration and integration of more sophisticated techniques represent an interesting direction for future developments.

1.2.4 Frameworks and Implementation Solutions

In dual-arm system applications, MoveIt² is typically used as the core for the system's kinematic and geometric management, as described by Shao *et al.* [13]. In this setup, MoveIt integrates native modules for inverse kinematics (KDL library) and geometric path planning (OMPL library [14]). Once a geometrically valid trajectory is computed, the framework delegates execution to low-level controllers by sending the interpolated waypoints via the `FollowJointTrajectory` action interface. In reality MoveIt can be also only used in virtue of the internal planning algorithms (like RRT-Connect [15]) and to generate feasible waypoints, that are safe and that maximize a certain functional; an example of how such framework can be used as generator of points in complex contexts is provided by Sepúlveda *et al.* [16], where the solver STOMP (Stochastic Trajectory Optimization for Motion Planning) has been used to generate trajectories taking account of needs of smoothness and of distance from the obstacles; in this case MoveIt generates the waypoints in the space that are provided as input to the PID controllers internal to the dual arm system. When an application requires complex and interdependent operational sequences, MoveIt shows some limits. To execute complex and structured tasks, Görner *et al.* [17] introduce the MoveIt Task Constructor (MTC) module. This abstraction allows breaking down a

²Throughout this work, we adopt the convention of referring to the ROS 2 motion planning framework as MoveIt. For the sake of clarity, all subsequent mentions of MoveIt refer to the MoveIt 2 version unless otherwise specified.

complex operation into more simple operations in hierarchical tree structures. This approach enables the management of parallel task execution in dual-arm systems or allow the scheduling of different tasks between the two manipulators. However, using MTC involves a steep learning curve, partially offsetting the rapid implementation benefits originally offered by MoveIt. In this thesis, the MoveIt framework was initially used to configure the multi-robot system and to test the ROS 2 interface from the moment that the robots had been just bought by the laboratory and therefore it was necessary in primis to create an architecture that allowed to prescind from the proprietary software, as detailed in Chapter 3. However, the standard MoveIt architecture offers limited structural flexibility when it deals with using more advanced schemes of control linked to feedback of force or of visual informations. Therefore, to achieve the full architectural flexibility required by the visual servoing and coordinated manipulation specifications, we opted to migrate to a custom control architecture developed natively in `ros2_control`. The transition toward ROS 2 is largely driven by the need to overcome the limitations of proprietary cobot interfaces, which often restrict the deployment of complex, reactive tasks. As analyzed in a recent survey on ROS 2 frameworks by Bonci *et al.* [18], implementing a modular architecture based on ROS 2 provides the necessary hardware abstraction and real-time communication capabilities. This middleware enables the creation of a control layer that integrates exteroceptive perception with advanced motion control, representing an essential requirement for deploying flexible and reactive robotic systems in dynamic environments.

1.3 Structure of the thesis

The thesis work is structured into the following chapters, the content of which is briefly illustrated below:

- Chapter 2 is dedicated to the mathematical and methodological treatment of the problem. Initially, the adopted control schemes will be illustrated, and particular attention will be paid to the use of the gradient projection method to allow the manipulators to perform multiple tasks using a hierarchical scheme.

-
- Chapter 3 describes the developed infrastructure and its validation. The architecture based on the ROS 2 middleware for hardware interfacing and synchronization of the two robotic arms will be illustrated. In parallel, a quantitative analysis of the experimental results obtained on the physical setup will be presented.
 - Chapter 4 gathers the conclusions of the work, summarizing the main contributions of the research and outlining the current limitations and possible future developments of the proposed architecture.

Chapter 2

Methodology

This chapter details the methodology developed to control the dual-arm robotic system. The control architecture integrates multiple strategies, tailored to the specific operational requirements of each manipulator to ensure safe collaboration in a shared workspace. The control framework was progressively built to assign specific roles to each arm. First, for the manipulator without a camera, whose primary task is environmental interaction, the Closed-Loop Inverse Kinematics (CLIK) algorithm was adopted. This baseline control law is used whenever it is necessary to move the arm along predefined operational space waypoints. Conversely, the manipulator equipped with an eye-in-hand camera relies on Image-Based Visual Servoing (IBVS) to track objects of interest, using visual features directly in the control loop. To guarantee safety and adaptability for the camera-equipped arm, the Projected Gradient Method (PGM) is integrated with the IBVS scheme. This technique exploits the system's kinematic redundancy, allowing the robot to evade spatial invasions (*e.g.*, from the other arm during collaborative maneuvers) without perturbing the primary visual tracking task. Finally, since physical collision avoidance does not prevent line-of-sight obstructions, a Quadratic Programming (QP) solver is introduced. By formulating occlusion avoidance as a constrained optimization problem, this approach ensures the target remains continuously visible during task execution. The remainder of this chapter is structured to reflect this logical progression. Section 2.1 introduces the differential kinematics and the CLIK algorithm. Section 2.2 describes the IBVS framework. Section 2.3 details the PGM formulation for reactive collision avoidance. Finally, Section 2.4

explores the constrained optimization approach for occlusion-free visual servoing. It is worth specifying that all the control laws implemented in this work rely on a resolved-velocity approach, a strategy detailed by Siciliano *et al.* [19]. This approach assumes that the internal controllers of the manipulators are sufficiently effective to accurately track the joint reference velocities generated by the control laws described in the following sections.

2.1 Differential Kinematics

To properly formulate the closed-loop kinematic inversion algorithm [19], it is first necessary to introduce the fundamental quantities describing the manipulator. Let $q \in \mathbb{R}^n$ denote the joint configuration vector, where n is the number of joints of the robot. The pose of the end-effector in the operational space is represented by the vector x , which encapsulates both position and parameterized orientation. The forward kinematics function defines the non-linear relationship mapping the joint space to the Cartesian space.

At the differential level, let $\dot{q} \in \mathbb{R}^n$ represent the joint velocities vector and $\dot{x}$ represent the Cartesian velocities vector. The geometric Jacobian matrix of the manipulator, denoted by J , represents the linear mapping from the joint velocities space to the Cartesian velocities space, expressed by the fundamental relation $\dot{x} = J\dot{q}$.

The closed-loop kinematic inversion algorithm is used to determine the joint velocities $\dot{q}$ necessary to ensure that the Cartesian space error ($\tilde{x}$) asymptotically tends to zero. In some cases, assuming that the internal joint controllers of the manipulators are sufficiently performant and capable of strictly tracking velocity references, this algorithm can be employed to control the manipulators in velocity. This control law was adopted solely in cases where there was no particular interest in implementing a more sophisticated control scheme for the manipulator, but only to follow spatial waypoints. To achieve this, the closed-loop inverse kinematics (CLIK) algorithm employed is of the first order and is based on the pseudo-inverse of the geometric Jacobian of the manipulator. Furthermore, assuming that the desired pose is constant, the control law is given by

$$\dot{q} = J^\dagger K \tilde{x}, \quad (2.1)$$

where $K \in \mathbb{R}^{6 \times 6}$ is a symmetric positive-definite gain matrix (typically a diagonal matrix), and $J^\dagger$ denotes the Moore-Penrose pseudo-inverse of the Jacobian¹. As detailed in [19], the pseudo-inverse extends matrix inversion to rectangular matrices. For redundant manipulators ($n > 6$), the Jacobian is a $6 \times n$ matrix and the differential kinematics problem admits infinite solutions. By adopting the right pseudo-inverse, analytically defined as $J^\dagger = J^T (JJ^T)^{-1}$, the redundancy is resolved by naturally selecting the unique joint velocity vector $\dot{q}$ that minimizes the Euclidean norm $\|\dot{q}\|$. For non-redundant manipulators ($n = 6$), assuming a non-singular configuration, this generalized formulation directly coincides with the standard matrix inverse J^{-1} .

Regarding the calculation of the Cartesian space error $\tilde{x}$, it was determined starting from the 4×4 homogeneous transformation matrices T_c and T_d , where T_c represents the current pose of the end-effector with respect to the base frame, while T_d represents its desired pose. It is possible to calculate the 4×4 homogeneous transformation matrix that describes the pose of the desired frame with respect to the current frame as

$$T_d^c = (T_c)^{-1} T_d \in SE(3). \quad (2.2)$$

As detailed by Lynch *et al.* [15], in order to extract the corresponding error vector in $\mathbb{R}^6$, exponential coordinates can be used for the representation of rigid body motion. Specifically, according to the Chasles-Mozzi theorem, every displacement of a rigid body can be described as a helical motion along an axis (screw axis) $\mathcal{S}$. The exponential coordinates of a transformation T are therefore defined by the vector $\mathcal{S}\theta \in \mathbb{R}^6$, where $\mathcal{S}$ is the normalized screw axis and θ is the magnitude of the displacement to be performed along and around this axis so that the two frames coincide.

While the matrix exponential maps the coordinates $\mathcal{S}\theta$ into the corresponding 4×4 homogeneous transformation matrix, the logarithmic map applied to the matrix T_d^c performs the inverse operation. By calculating the matrix logarithm of T_d^c , it is possible to directly extract the 6-element vector $\mathcal{S}\theta$, which constitutes the exact representation of the Cartesian space error $\tilde{x}$.

¹Hereafter, the superscript $\dagger$ denotes the pseudo-inverse of a matrix.

2.2 Image Based Visual Servoing

Image-Based Visual Servoing (IBVS) [19], is based on the definition of a task error relying on features directly extracted from the image plane. Let $s \in \mathbb{R}^m$ denote the feature vector of dimension m . In the considered application, this vector represents the normalized image coordinates of the centroid of the object of interest; therefore, $m = 2$ and the vector is defined as

$$s = \begin{bmatrix} X \\ Y \end{bmatrix}. \quad (2.3)$$

The control scheme relies on the differential relationship between the time derivative of the feature vector, $\dot{s} \in \mathbb{R}^2$, and the spatial velocity of the camera. Let $v_c^c \in \mathbb{R}^6$ denote the absolute velocity of the camera expressed with respect to a frame attached to the camera itself. The kinematic mapping is given by

$$\dot{s} = L_s v_c^c, \quad (2.4)$$

where $L_s \in \mathbb{R}^{m \times 6}$ is the interaction matrix. This matrix defines the linear transformation that relates the absolute velocity of the camera v_c^c to the velocity on the image plane $\dot{s}$, under the assumption that the observed object is stationary. Specifically, for a single point feature with normalized image coordinates $s = [X, Y]^T$, the interaction matrix L_s is a 2×6 matrix that depends on the depth $Z \in \mathbb{R}^+$ of the point in the space relative to the camera frame, analytically expressed as

$$L_s = \begin{bmatrix} -\frac{1}{Z} & 0 & \frac{X}{Z} & XY & -(1+X^2) & Y \\ 0 & -\frac{1}{Z} & \frac{Y}{Z} & 1+Y^2 & -XY & -X \end{bmatrix}. \quad (2.5)$$

To perform the visual servoing task, let $s_d \in \mathbb{R}^2$ be the constant desired feature vector. The error in the image space is defined as $e_s = s_d - s \in \mathbb{R}^2$. By taking the time derivative of the error and assuming that the desired feature velocity is zero ($\dot{s}_d = 0$), we obtain

$$\dot{e}_s = -\dot{s} = -L_s v_c^c = -L_s \begin{bmatrix} R_c^T & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & R_c^T \end{bmatrix} J_c \dot{q} = -J_L \dot{q}, \quad (2.6)$$

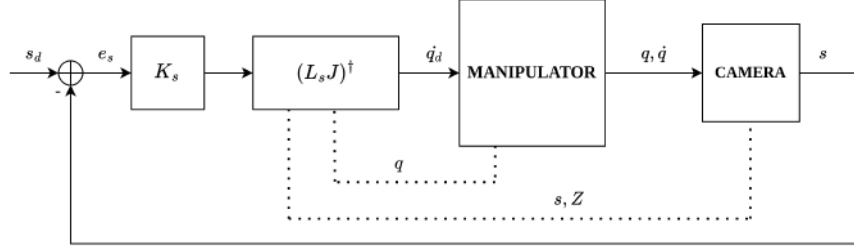


Figure 2.1 – IBVS Control Scheme.

where $\dot{q} \in \mathbb{R}^n$ is the joint velocities vector of the manipulator, and $J_c \in \mathbb{R}^{6 \times n}$ is the geometric Jacobian of the camera evaluated with respect to the base frame. The matrix $R_c \in SO(3)$ is the 3×3 rotation matrix expressing the orientation of the camera frame with respect to the base frame; consequently, the 6×6 block diagonal matrix containing R_c^T performs the necessary coordinate transformation of the spatial velocity from the base frame to the camera frame. The resulting matrix $J_L \in \mathbb{R}^{2 \times n}$ defines the overall transformation mapping the manipulator's joint velocities directly to the image feature velocities. Since the interaction matrix L_s for a single point is rectangular (2×6), the resulting matrix J_L is generally not square. Therefore, adopting a control law based on the Moore-Penrose pseudo-inverse, it is possible to define the reference joint velocities $\dot{q}_d \in \mathbb{R}^n$ as

$$\dot{q}_d = J_L^\dagger K_s e_s \quad , \quad (2.7)$$

where $K_s \in \mathbb{R}^{2 \times 2}$ is a symmetric, positive-definite gain matrix, and $J_L^\dagger$ denotes the pseudo-inverse of the matrix J_L . With this control law, the image space error e_s asymptotically converges to zero. The complete IBVS control scheme is depicted in Fig. 2.1.

2.3 Projected Gradient Method

The image-based error e_s and the matrix J_L , relating the joint velocities to the image feature velocities, were introduced in Section 2.2. Since the feature vector considered has dimension $m = 2$, while the manipulator carrying the camera has n degrees of freedom, only 2 of the n available degrees of freedom are constrained by the primary visual task, provided that $n > 2$. Under this condition, and assuming J_L full row rank, the manipulator is said to

be functionally redundant with respect to the task, and the associated inverse differential kinematics problem admits infinite solutions. The general solution can be written as

$$\dot{q} = k_{r1} J_L^\dagger e_s + \left(I_n - J_L^\dagger J_L \right) \dot{q}_0, \quad (2.8)$$

where k_{r1} is a positive proportional gain and $\dot{q}_0 \in \mathbb{R}^n$ is an arbitrary joint velocity that, once projected onto the null space of J_L through the projector $I_n - J_L^\dagger J_L$, does not affect the execution of the primary task.

The projected gradient method (PGM) exploits this redundancy by choosing $\dot{q}_0$ so as to minimize a scalar, and generally non-negative, cost function $H(q)$ that encodes secondary requirements on the robot motion. Specifically,

$$\dot{q}_0 = -k_{r2} \nabla H, \quad (2.9)$$

where k_{r2} is a positive gain and ∇H denotes the gradient of H with respect to the joint configuration². Substituting in Equation (2.8), and denoting with the subscript c the camera-carrying manipulator considered in this section, the resulting control law is

$$\dot{q}_c = k_{r1} J_L^\dagger e_s - k_{r2} \left(I_n - J_L^\dagger J_L \right) \nabla H. \quad (2.10)$$

This approach is not only versatile in satisfying multiple system requirements simultaneously, but also allows a strict hierarchy to be established among the different tasks, preventing mutual interference. The corresponding control scheme is depicted in Fig. 2.2.

The analytical proof is presented by Siciliano, Slotine *et al.* [20] showing that this control law imposes a hierarchical decoupling, where the primary task is unaffected by the secondary gradient minimization. This theoretical property is, moreover, fully corroborated by the experimental results discussed in the subsequent sections.

In general, the cost function H from which the gradient is derived can take various forms. Following the formulation proposed by Selvaggio *et al.* [8], the cost functional is composed of 3 independent contributions, designed to address 3 distinct, and not necessar-

²That is, $\nabla H = [\partial H / \partial q_1, \dots, \partial H / \partial q_n]^\top$.

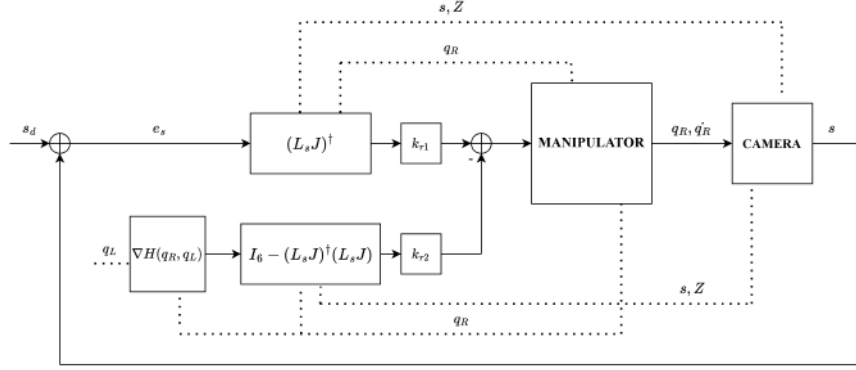


Figure 2.2 – PGM Visual Servoing Control Scheme.

ily complementary, requirements

$$H = h_A + h_B + h_C. \quad (2.11)$$

Specifically,

$$h_A(q) = \rho_J \sum_{i=1}^n \left[e^{-\alpha_J(q_{i,\max} - q_i)} (q_{i,\max} - q_i)^{-\beta_J} + e^{-\alpha_J(q_i - q_{i,\min})} (q_i - q_{i,\min})^{-\beta_J} \right] \quad (2.12)$$

is the contribution related to the proximity to the robot's joint limits. By minimizing this cost function, the robot attempts to stay safely away from these bounds. This type of contribution is essential when incorporating a cost related to the distance from a secondary manipulator, since it prevents the controlled manipulator from reaching the workspace limits in its attempt to maximize such distance. Here, $q_{i,\max}$ and $q_{i,\min}$ represent the upper and lower limits of the i -th joint, respectively, while ρ_J , α_J , and β_J are appropriately chosen positive tuning constants.

The second contribution was introduced to prevent the manipulator from reaching singular configurations. By minimizing this functional, the determinant of the geometric Jacobian is maximized

$$h_B(q) = \rho_S e^{-\alpha_S \det(\Gamma J_c(q))} (\det(\Gamma J_c(q)))^{-\beta_S}, \quad (2.13)$$

where $J_c(q)$ is the geometric Jacobian, ρ_S , α_S , and β_S are chosen positive constants, and Γ

is a positive-definite diagonal weighting matrix.

Finally, the third contribution is the only one that is a function of the joint configurations of both manipulators, and it is aimed at managing collision avoidance. It is assumed that the collision volumes of the manipulators are approximated by bounding spheres, although other, more or less conservative, geometric choices are possible. Let S_A and S_B denote the total number of spheres assigned to the first and second manipulator, respectively, and let d_{ij} denote the Euclidean distance between the center of the i -th and the j -th sphere. The corresponding cost functional is defined as

$$h_C(q_A, q_B) = \rho_C \sum_{i=1}^{S_A} \sum_{j=1}^{S_B} e^{-\alpha_C d_{ij}} d_{ij}^{-\beta_C}, \quad (2.14)$$

where ρ_C , α_C , and β_C are properly defined positive constants.

2.4 Occlusion-free Visual Servoing

One of the main issues in visual servoing relates to the presence of objects in the scene that can occlude the field of view of the camera. Solving such a problem becomes more relevant if only one robot has a camera; specific variations to the classical visual servoing control scheme are needed in order to take into account occlusions and in order to make the robot react to them properly. A possible way to deal with occlusions in visual servoing is imposing a constrained optimization problem in the joint space of both the manipulators, inserting proper geometric constraints in order to avoid occlusions [10, 21].

Assuming we have a simple obstacle, a parallelepiped, it's possible to consider two of its vertices in the image plane, p_1 and p_2 , and formulate a geometric constraint based on the joint velocities; more specifically, if we consider a point of interest p_t in the image space, then avoiding collisions means that the point p_t and the segment $\overline{p_1 p_2}$ have a distance different from zero. Such reasoning can be reiterated for all the edges of the object. Let's consider a generic point p_s on the segment, defined as

$$p_s = p_1 + \lambda(p_2 - p_1), \quad \lambda \in [0, 1]. \quad (2.15)$$

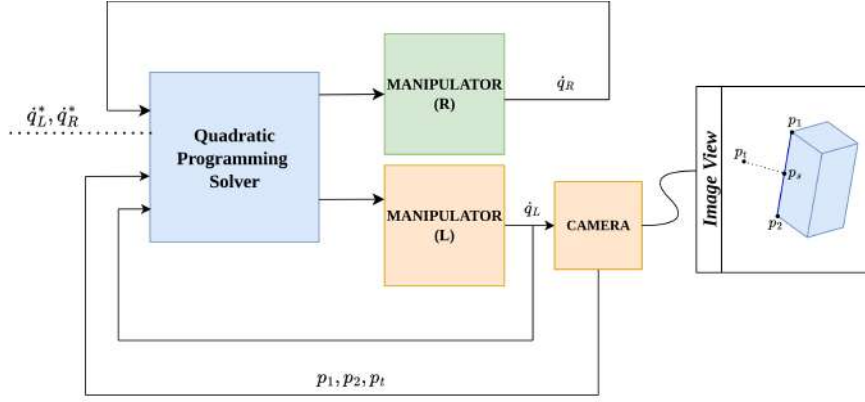


Figure 2.3 – Occlusion-free Visual Servoing Scheme.

Then such a requirement can be translated into the following relation order:

$$\|p_t - p_s\|^2 - t_b(p_t - p_s)^T(\dot{p}_s - \dot{p}_t) \geq 0, \quad (2.16)$$

or also as

$$\|p_t - p_s\| \geq t_b \frac{(p_t - p_s)^T}{\|p_t - p_s\|} (\dot{p}_s - \dot{p}_t); \quad (2.17)$$

a clearer interpretation of this constraint can be obtained by referring to the work of Zanchettin and Rocco [21], who employed the same formulation to address collision avoidance between a manipulator link and a physical obstacle. In Equation (2.17), the term $\frac{(p_t - p_s)^T}{\|p_t - p_s\|} (\dot{p}_s - \dot{p}_t)$ is the relative velocity between a point of the segment and the point of interest projected on the segment connecting them; this term is multiplied by a parameter t_b , which is a design parameter related to the braking time of the manipulator. Then, if Equation (2.16) is satisfied, the collision will happen after t_b , and then it is avoided. This can be translated into the field of visual servoing by considering the collision as an occlusion due to the presence of the object in the camera field of view (FoV). Nicolis *et al.* [10] detail the mathematical derivations to obtain this relation in affine form. To define the constraints, the authors substitute Equation (2.15) into Equation (2.16), and this yields a second-order polynomial inequality in the parameter λ

$$(\alpha + \alpha')\lambda^2 + (\beta + \beta')\lambda + (\gamma + \gamma') \geq 0, \quad \forall \lambda \in [0, 1]. \quad (2.18)$$

where the coefficients depend on the current image feature coordinates and their velocities. Since Equation (2.16) must hold for every point of the segment (*i.e.* for every $\lambda \in [0, 1]$) it is sufficient to enforce it at the single most critical point, namely the one at which the left-hand side of the parabola in Equation (2.18) attains its minimum value, denoted $\bar{\lambda}$. This minimum can be computed at each control cycle from the current, measured feature positions and velocities, since these enter Equation (2.16) only through the known coefficients of the resulting parabola in λ . Once $\bar{\lambda}$ is found, the corresponding point on the segment can be treated as fixed, and the original constraint reduces to a single scalar inequality, which still depends on the velocities $\dot{p}_1$, $\dot{p}_2$, and $\dot{p}_t$ of the relevant feature points.

These velocities, however, are related to the actual decision variables of the control problem, namely the joint velocities of the two manipulators, through the differential kinematics of the observed points. The constraint must therefore be mapped from the image feature domain to the robot joint space, exploiting the relationship between the derivative of the feature vector and the joint velocity. For example, for the feature point of the object p_1 we have

$$\dot{p}_1 = P_{c,1} \dot{q}_c = L_1 J_c^c \dot{q}_c \quad , \quad (2.19)$$

where L_1 is the interaction matrix of the point p_1 , $\dot{q}_c$ are the joint velocities of the robot with the camera (that should avoid the collision) and J_c^c is the camera Jacobian expressed in the camera frame. Finally, if we consider the point of interest p_t as the tool point of the other manipulator, the following relation must be taken into account

$$\dot{p}_t = P_t \dot{q} = L_t J_c^c \dot{q}_c + L_t \begin{bmatrix} -R^c & 0 \\ 0 & 0 \end{bmatrix} J_t \dot{q}_t \quad , \quad (2.20)$$

where L_t is the interaction matrix of p_t , R^c is the rotation matrix from the world frame to the camera frame and $\dot{q}_t$ are the joint velocities of the other manipulator.

Since these kinematic relations are linear in the joint velocities, substituting them into the scalar inequality obtained at $\bar{\lambda}$ turns the constraint, originally expressed in terms of image feature velocities, into an affine inequality expressed directly in terms of the joint velocities $\dot{q} = [\dot{q}_c^T \ \dot{q}_t^T]^T$. Once this substitution has been carried out for the points p_1, p_2, p_t ,

Equation (2.16) can be cast as a linear inequality in the joint velocities $\dot{q}$ (see, again, [10] for further details), and the optimization problem can be formulated in the following form

$$\begin{aligned} \min_{\dot{q}} \quad & ||\dot{q} - \dot{q}^*|| \\ \text{s.t.} \quad & E\dot{q} \geq g \quad , \end{aligned} \tag{2.21}$$

where $\dot{q}^*$ are the desired joint velocities. In Figure 2.3 a conceptual diagram of the control mode that was employed is depicted. We can notice how the control paradigm has changed: while above we employed functional redundancy, here we are solving a constrained optimization problem, and therefore the velocities may deviate from the desired ones as long as the constraints are satisfied. Moreover, in this configuration, occlusion avoidance is treated as a hard constraint, while the primary visual servoing task is incorporated into the cost function to be minimized as a soft objective. This may imply that when an occlusion becomes imminent, the QP solver could generate joint velocities that are very different from those currently being followed, resulting in abrupt motions and discontinuities in the reference velocities profile, but still ensuring that the geometric constraints are satisfied.

Chapter 3

Experiments and Results

3.1 Experimental setup

The experimental validation was conducted on multiple bimanual setups to demonstrate the versatility of the proposed control architecture. The first setup is based on the use of two Neura LARA 8 collaborative manipulators, characterized by 6 degrees of freedom (DoF) kinematics. Both robots have a nominal payload of 8 kg and a maximum reach of 1300 mm [22]. In order to enable the execution of cooperative pick-and-place and visual servoing tasks, the flanges of both arms have been equipped with dedicated sensors and end-effectors. Specifically, the end-effector is a prototype gripper developed in-house at the host institution’s Laboratory of Robotics (RoboLab) shown in Figure 3.1b, whose control interface is managed asynchronously via socket communication. An Intel RealSense D435i depth camera has been integrated for visual feedback acquisition, rigidly mounted on the robot’s wrist in an eye-in-hand configuration (Figure 3.1a). Considering the nominal payloads of the robots, the mass of these two tools does not significantly alter the operating conditions of the robots. The complete configuration of the two manipulators is depicted in Figure 3.2.

The first operation performed on the system was to interface the two manipulators with the ROS 2 framework [1], utilizing the official packages provided by the manufacturer [23]. This architectural choice was strictly motivated by the limitations of the proprietary C++ and Python APIs [24, 25] offered by Neura Robotics. Indeed, while these interfaces provide



Figure 3.1 – Camera (a) and gripper (b) mounted on the two manipulators.



Figure 3.2 – Setup of the two manipulators.

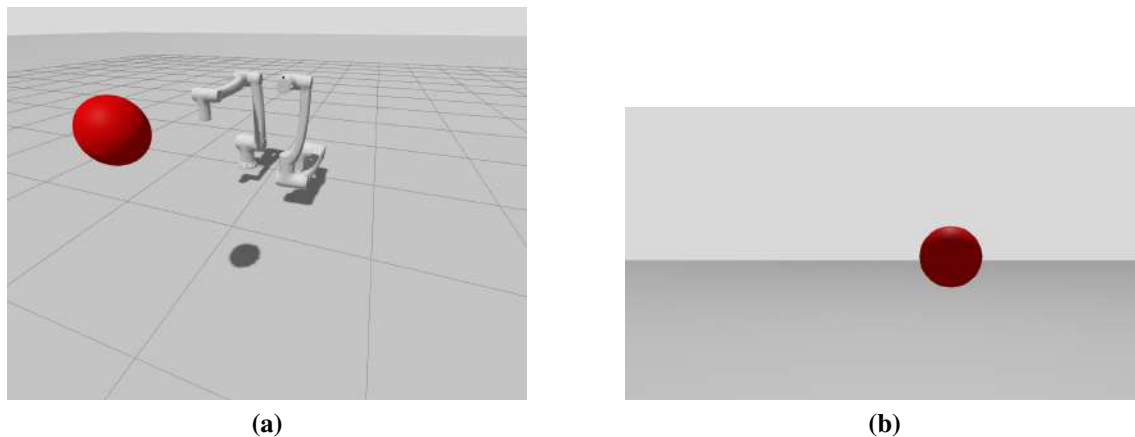


Figure 3.3 – Gazebo environment (a) and simulated camera image (b).

basic functionalities such as reading joint positions and velocities, and solving standard forward and inverse kinematics, they are inherently limiting for advanced research purposes. Specifically, they do not provide access to essential differential kinematic quantities, such as the geometric Jacobian matrix, nor do they support the definition of custom frames, such as the one associated with the camera mounted on the robot's end-effector. Consequently, they do not allow for the implementation of more sophisticated control schemes, such as those relying on visual or also force feedback, and they fail to provide the synchronous communication required for bimanual coordination. Simulation and real-world validation were orchestrated using a workstation equipped with an AMD Ryzen 9 7900 12-Core Processor and 61 GB of RAM. The adopted Operating System is Ubuntu 24.04.3 LTS. The communication between the centralized ROS 2 workstation and the manipulators is established via an Ethernet connection, with all devices configured within the same Local Area Network (LAN). Conversely, the Intel RealSense depth camera interfaces directly with the workstation through a USB connection. The software infrastructure is built upon the ROS 2 Jazzy Jalisco distribution [31]. Furthermore, given the risk of collisions between the two robots, a preliminary kinematic validation of each control law was mandatory and was conducted in simulation using Gazebo Sim [26], version 8.10.0. Specifically, a suitable environment was created in order to safely test the Visual Servoing tasks. Figure 3.3a shows the Gazebo simulation environment, while Figure 3.3b shows the image captured by the simulated camera.

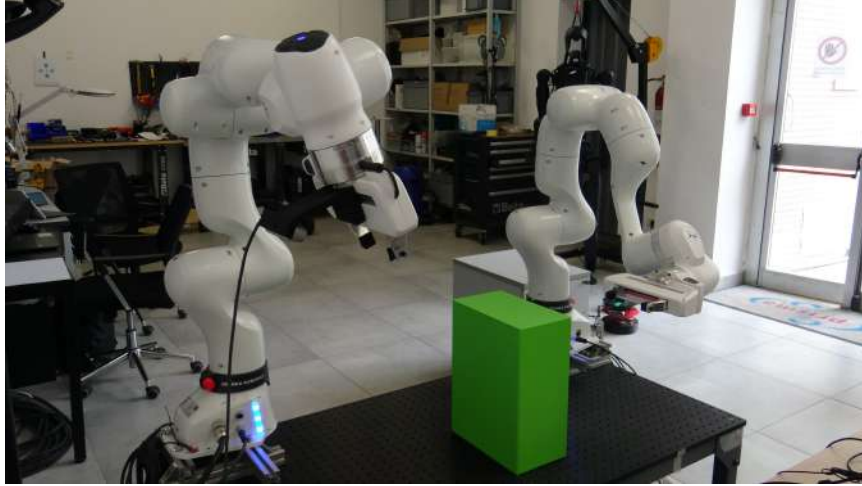


Figure 3.4 – Experimental setup of the dual Franka Emika Panda robotic system.

Finally, it is worth noting that the latest experimental validations, concerning the algorithmic extension for the occlusion avoidance problem, were conducted on a different bimanual setup. This secondary configuration comprises two Franka Emika Panda collaborative manipulators (Figure 3.4).

3.1.1 Driver modification

In order to control both robots using the same workstation, it was necessary to modify the provided drivers to allow compatibility between the hardware and the ROS 2 framework. The standard architecture of `ros2_control` [32] manages the hardware abstraction through the Resource Manager (RM), which exposes the physical interfaces to the Controller Manager (CM). The RM exploits the `pluginlib` library to dynamically load at runtime an instance of the abstract class `hardware_interface::SystemInterface`. The latter defines how the hardware is exposed to the CM: of essential importance from this point of view are the definitions of the `read()` and `write()` methods, through which it is possible to read the joint states and send commands to the actuators. The original implementation of the driver presented two architectural limitations incompatible with the thesis objectives, making the following modifications necessary:

1. **Multi robot support:** The `SystemInterface` class was modified to accept the serial number of the manipulator as an initialization parameter. In this way, it is possible

to instantiate two independent objects within the same RM, guaranteeing the simultaneous management of the dual-arm system without conflicts.

2. **Integration of velocity interfaces:** The base driver exclusively allowed sending poses in the joint or Cartesian space. Exploiting the proprietary C++ APIs of Neura Robotics, the class was modified to expose velocity command interfaces.

3.2 MoveIt Integration

To implement a first centralized control strategy solving *a priori* the problem of collision avoidance, a dedicated MoveIt [33] package was developed for the control of both robots simultaneously. To this end, the *MoveIt Setup Assistant* was utilized. Given that the Neura multi-robot system is not natively supported, this tool was necessary for the generation of the Semantic Robot Description Format (SRDF), fundamental for MoveIt since in it are defined the end-effectors, the kinematic chains (planning groups) and, in particular, the Allowed Collision Matrix (ACM), necessary for the computational optimization of the interference checks. The software architecture of the framework is built around the `move_group` node, which exposes ROS 2 services and actions for motion planning. The interface relies on an Action-based client–server architecture. The Trajectory Execution Manager within MoveIt, configured through the `MoveItSimpleControllerManager` plugin, acts as an Action Client and sends the planned waypoints via the `FollowJointTrajectory` action.

The corresponding Action Server is provided by the `joint_trajectory_controller` of the `ros2_control` framework, which is responsible for low-level joint control and trajectory execution. To adapt the infrastructure to the dual-arm setup, the SRDF generated for the single LARA 8 was manually modified, adding in the file the joints of the second robot and combining the joints of the two manipulators into a single composite planning group. This makes it so that the MoveIt planner will automatically avoid (self)-collisions, given that the multirobot system is conceived as a *unicum*. The adopted motion planner is based on the Open Motion Planning Library (OMPL) [14], although other choices are possible (*e.g.*, Pilz Industrial Motion Planner), and the specific planning algorithm employed was RRT-Connect, which is particularly suited for high-dimensional configuration

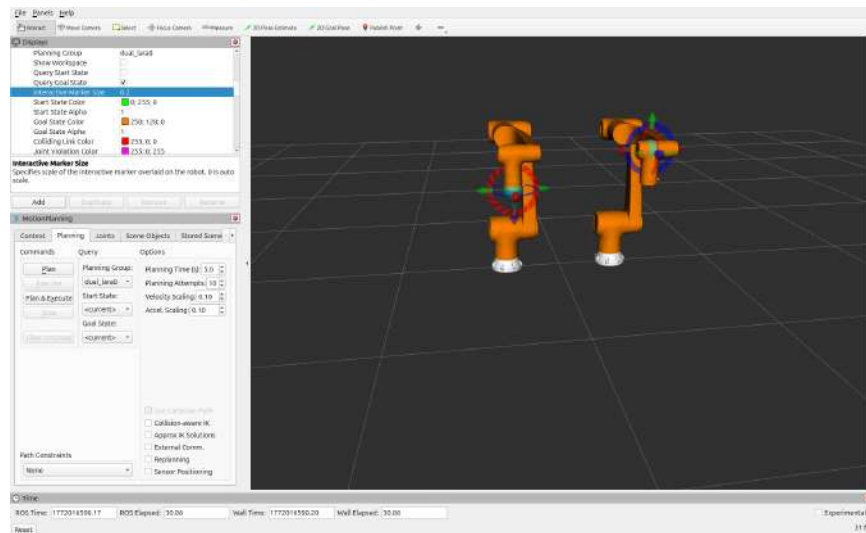


Figure 3.5 – Neura LARA 8 Manipulators in RViz.

spaces, as in this case. Finally, to guarantee the coherence between the simulation and the real workspace, the Planning Scene was enriched with geometric models representing the physical obstacles. This representation of the environment, manageable both via the graphical interface of RViz and via C++ API, allows to further constrain the free space available to the planning algorithms.

3.2.1 Validation of the MoveIt Motion Planner

Following the generation of the correct SRDF, a kinematic simulation of the dual arm system was performed within the RViz environment. The abstraction of the hardware components was achieved using the `mock_components/GenericSystem` plugin. Initially, the manipulators were controlled using the MoveIt plugin for RViz (see Figure 3.5). In order for the simulated environment to be faithful to the actual workspace of the multirobot system in the laboratory, the planning scene was enriched by including obstacles (see Figure 3.6), so that the generated trajectories were safe and to speed up the transition from the simulated environment to real world validation. This interface also makes it possible to separate the trajectory planning and execution phases, allowing the generated path to be viewed in advance and its feasibility to be verified. Following the testing phase, real world validation was carried out. Within the Unified Robot Description Format (URDF) file in-

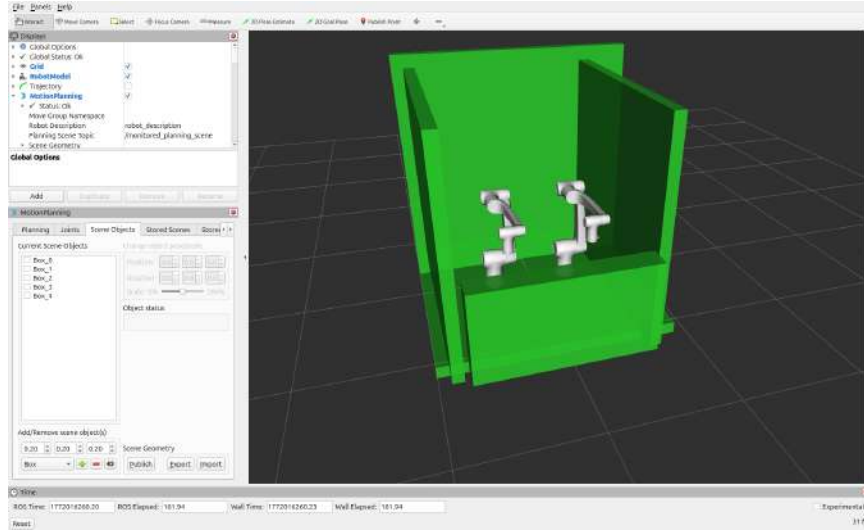


Figure 3.6 – Planning Scene in RViz.

side the `ros2_control` block, the `mock_components` plugin was replaced by the previous hardware interface, modified to allow simultaneous communication with both robots. Two poses were commanded in the operational space, contained within the shared workspace of the manipulators, to test the planning and collision avoidance capabilities. The inverse kinematics resolution was delegated to MoveIt, which uses the numerical solver provided by the KDL (Kinematics and Dynamics Library) plugin. In Figure 3.7 is possible to see the movements executed by the manipulators without collisions, and the corresponding poses assigned through the GUI.

3.3 Preliminary Validation via Vision-Guided Grasping

Since the ultimate goal of the thesis work consists in the implementation of control schemes based on visual feedback (visual servoing), the architecture developed in MoveIt was initially validated on a demonstrative pick and place task. While the first arm, equipped with the vision sensor, estimates the pose of the target in Cartesian space, the second arm, equipped with a gripper, executes the approach trajectory, the grasping phase, and the return to the initial configuration. Since both manipulators are part of the same planning group, and the manipulator with the camera can be considered as an obstacle in this context, the MoveIt framework guarantees the generation of intrinsically safe trajectories.

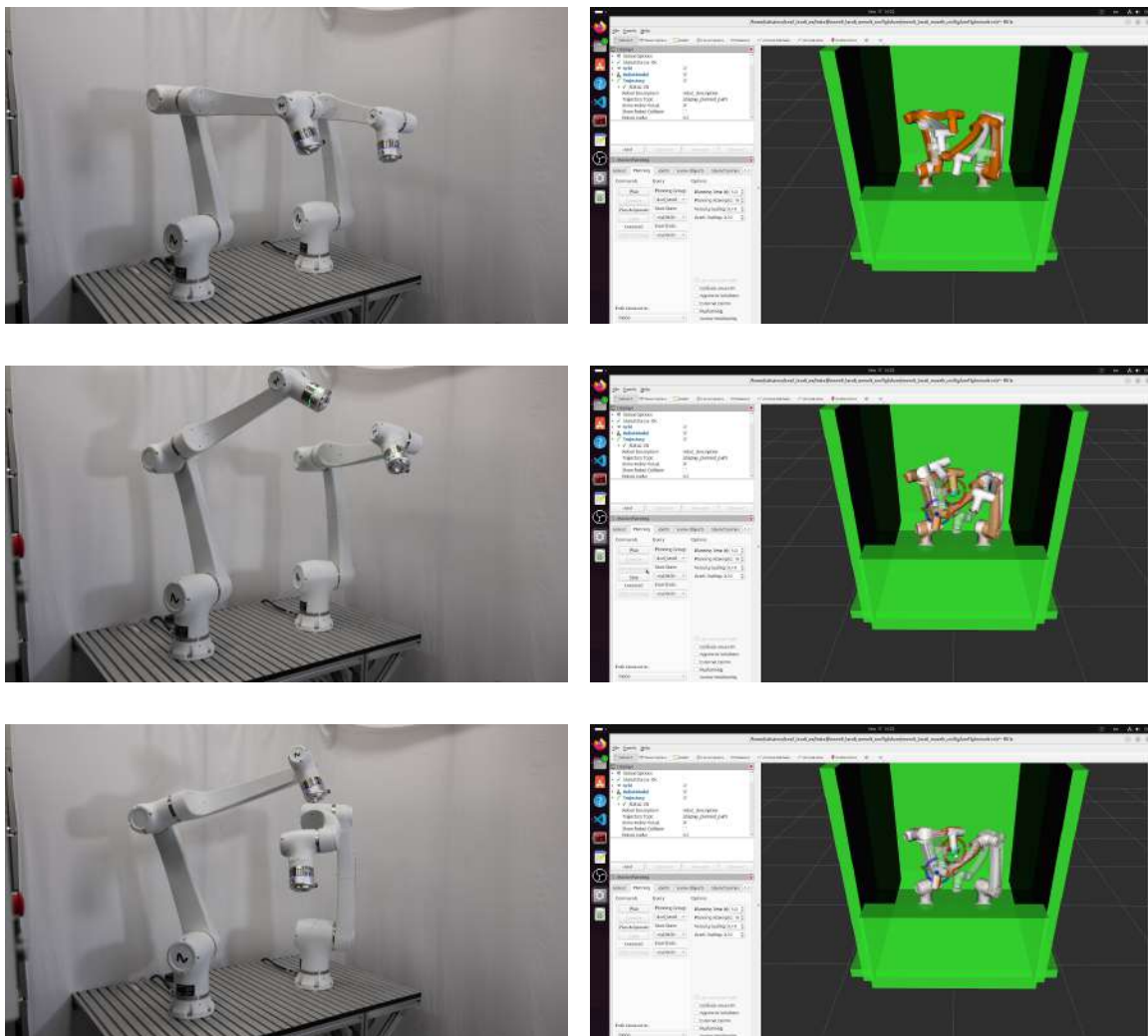


Figure 3.7 – Example of usage of the MoveIt plugin in RViz GUI.



Figure 3.8 – Yolo Model detection results.

3.3.1 YOLOv8 Model

To enable the camera-equipped robot to obtain visual information from the environment, a node based on a YOLOv8 model [27] has been implemented. Specifically, the Neural network underwent a tuning process for the recognition of a specific target (an apple) within the laboratory. The training was conducted utilizing an annotated dataset extracted from the Roboflow platform [34]. The annotations were not provided as simple bounding boxes, but rather through polygon annotations. This allows the model to not only identify the object and define a bounding box but also perform a segmentation of the same as is shown in Figure 3.8.

This neural network is used by a ROS 2 service (`/object_pose_service`) to publish a frame attached to the target object. Once the centroid of the mask on the image plane has been calculated (using the first-order moments of the identified image region), the extraction of the corresponding spatial value from the aligned depth image allows obtaining the 3D position of the object in Cartesian space. In particular, the RGB data and the depth map are published on the topics `D435i/color/image_raw` and `/D435i/aligned_depth_to_color/image_raw`, and to manage the consistency of the messages received, the `message_filters` package [35] was used to synchronize the two message streams and perform processing on messages with the same timestamp. Secondly, information is retrieved by querying the `/tf` topic, as the estimated pose refers to the camera frame and had to be referenced to the fixed frame of the world. The architecture is depicted in

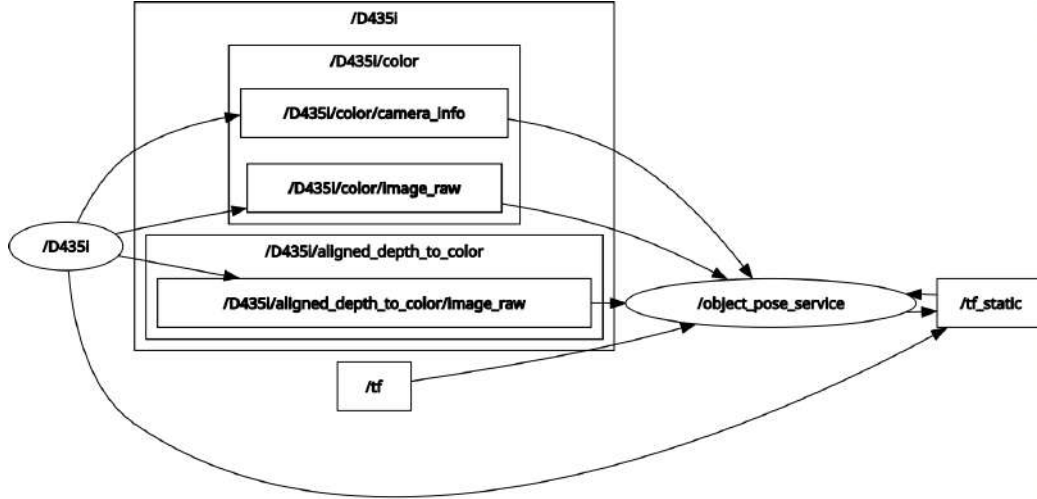


Figure 3.9 – Nodes and topics architecture of the detection task

Figure 3.9 . Particular attention was paid to defining the orientation of the frame assigned to the target object in order to prevent the manipulator used for gripping from occluding the camera’s field of view during the maneuver. The line-of-sight vector connecting the optical center of the camera to the target was calculated. By constraining one of the axes of the target frame to align with this vector, the remaining two axes were derived to form an orthonormal triplet. In this way, by forcing the manipulator’s end-effector to align with the plane normal to the line-of-sight, the approach trajectory avoids invading the camera’s visual volume. The result is shown in Figure 3.10.

3.3.2 Grasping Sequence

Following the invocation of the service and the subsequent definition of the frame integral with the target, it is possible to calculate the operating poses for the grasping manipulator. The task sequence has been structured into distinct kinematic phases. Initially, the manipulator reaches a pre-grasp pose (Figure 3.11b), aligning the end-effector with the calculated approach vector and activating the gripper opening. Subsequently, to ensure that the orientation remains unchanged during the approach to the object, the latter is performed by imposing a Cartesian path (Figure 3.11c). After closing the gripper, the robot returns to its

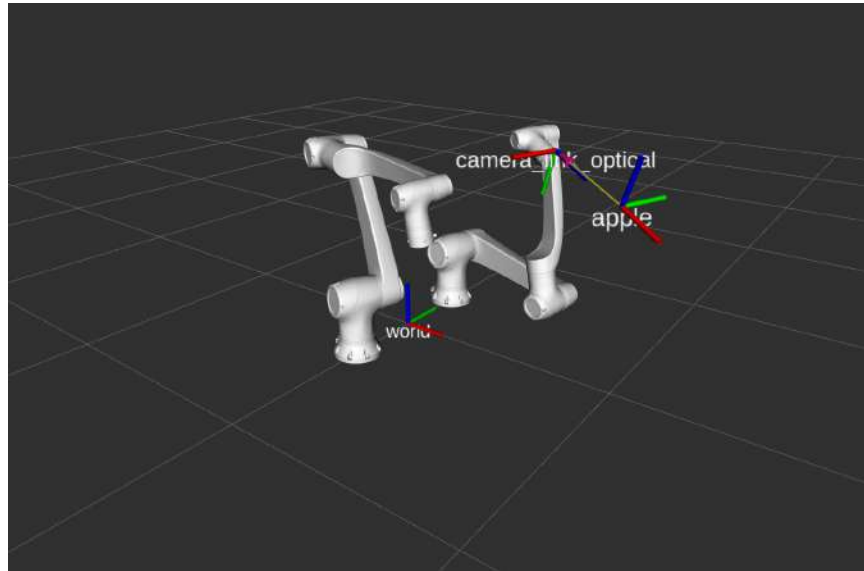


Figure 3.10 – Object’s frame.

initial starting position (Figure 3.11d-3.11a). The entire pipeline for calculating waypoints and executing trajectories was developed by implementing a node based on the C++ APIs provided by the MoveIt framework.

3.4 Advantages and Limitations of MoveIt

The implementation of the architecture based on MoveIt highlighted specific advantages and limitations.

Advantages:

- **Native collision avoidance:** the *a priori* resolution of the self-collision problem within the shared workspace drastically sped up the time to correctly control the multi-robot system.
- **Seamless sim-to-real transition:** once the proprietary driver was appropriately modified to allow simultaneous communication with both robots, the transition from simulation to real world validation was immediate.

Drawbacks:

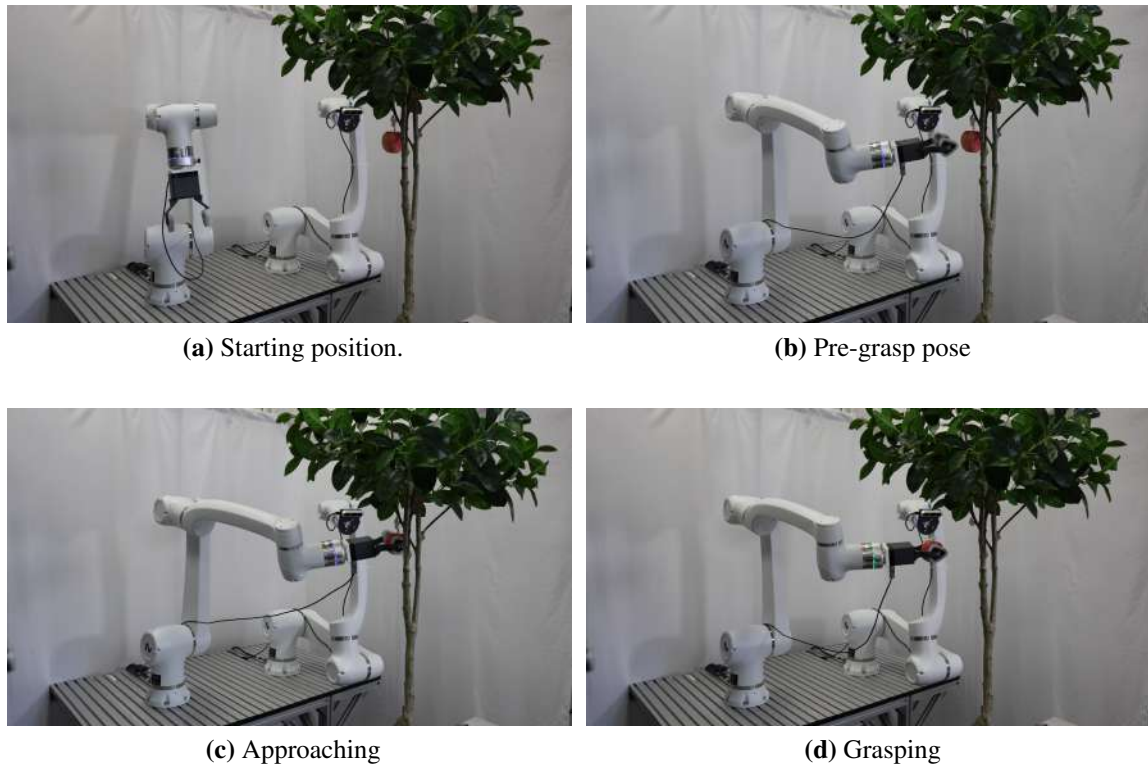


Figure 3.11 – Grasping pipeline using MoveIt.

- **Computational overhead:** the framework’s architecture introduces non-deterministic delays that are incompatible with high-frequency control schemes, such as visual servoing tasks. Although the `moveit_servo` [36] package allows velocity control of the manipulators, it not only introduces a higher computational load compared to other solutions, but the learning curve for such tools is also quite high.
- **Trajectory generation:** the default planners provided by OMPL rely on randomized sampling techniques. While effective for obstacle avoidance, this approach can lead to unpredictable paths in the joint space.

In the end MoveIt was a useful tool for the rapid prototyping of the robotic setup. Delegating the complex management of interferences to the framework allowed focusing development efforts on the construction and calibration of the visual perception model. However, the migration towards an implementation directly in `ros2_control` is necessary when one wants to use more advanced control techniques.

3.5 Custom Control Architecture

Given the limitations of MoveIt to test specific control laws on the two robots to perform a collaborative task, it was chosen to migrate towards the native `ros2_control` framework. In particular, as mentioned in Chapter 2 the approach used is of the resolved-velocity type [19], under the hypothesis that the internal controllers of the two manipulators perfectly track velocity references, both in the joint space and in the operational space. An *ad hoc* package was developed in which the two manipulators are initialized by loading controllers of the `velocity_controllers/JointGroupVelocityController` type. Given that the control laws to be implemented are based on visual feedback information, the preliminary test phase required the creation of a simulation environment in Gazebo (necessary having to simulate a depth camera), to then move to real-world validation. Finally, since it is necessary to perform operations related to the calculation of the Jacobian, the geometric transformations between the various frames of the manipulators and in general algebraic manipulations, the C++ library Pinocchio [28] was chosen to compute the dynamics of the manipulators under examination. Such library allows parsing the URDF file to instantiate distinct objects: not only objects dedicated to the resolution of the kinematics of the robot, but also specific objects that take into account the visual meshes and the collision volumes specified within the descriptive file of the manipulator. Furthermore, being Pinocchio a library heavily based on C++ templates, it allows an integration with nonlinear optimization frameworks such as CasADi [29], a characteristic that will be particularly useful in the following. For consistency with Chapter 2, the joint velocities of the camera-equipped manipulator continue to be denoted by $\dot{q}_c$ throughout this chapter, while for the second manipulator the joint velocities are denoted by $\dot{q}_t$.

3.5.1 Visual Servoing

Initially, we tested the speed control capability of only the manipulator equipped with an eye-in-hand camera; in the following section, we will describe how the system was extended to enable simultaneous and safe control of both manipulators. The strategy adopted for the visual servo task is based on an image-based visual servo control scheme. Specifi-

T_{a2}	e_{∞}	K_v
6s	2×10^{-3}	I_6

Table 3.1 – IBVS Results.

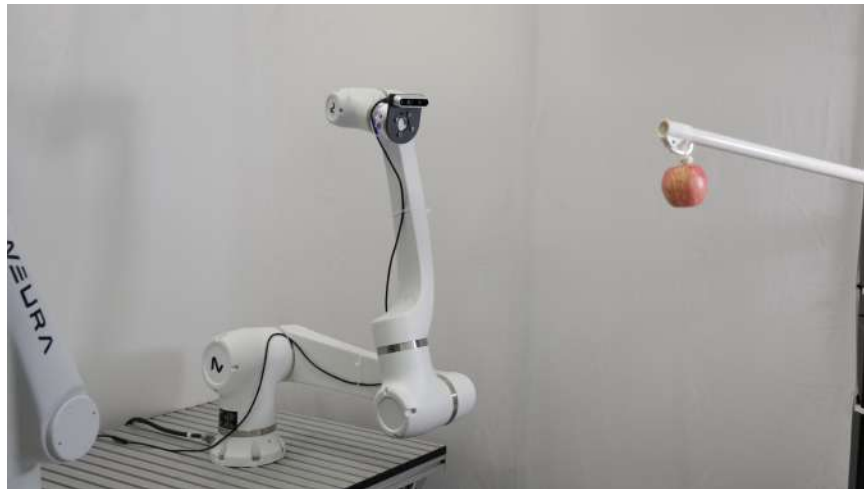
cally, joint velocities are calculated based on an error expressed in terms of visual features. The following control law is adopted

$$\dot{q}_c = K_v (L_s J_c)^\dagger (s_d - s) \quad (3.1)$$

In visual inspection, the feature vector can contain a variety of quantities, but in this case, the feature vector s is given by the normalized coordinates of the object's centroid in the image plane. The goal of the inspection is to adjust these coordinates to zero, so that the object remains centered in the image. Note that the task space has a dimension of 2. Since the manipulator has 6 degrees of freedom, the system is functionally redundant with respect to the assigned task; this redundancy will be exploited later for the simultaneous control of the two manipulators.

3.5.1.1 Results

The gain matrix is diagonal and contains identical gains; as a result, the projection of the centroid onto the image plane will follow a straight-line path. As illustrated in Figure 3.13, the centroid coordinates converge asymptotically to zero. During the experimental phase the tracked object was moved such that only one coordinate at a time varied significantly. Numerical results are presented in Table 3.1, where, here and in what follows, T_{a2} is the 2% settling time, e_{∞} is the steady-state error (since we have considered several movements of the target object, we took the final steady-state value). As illustrated in Figure 3.12, the object was manually displaced via a rigid linkage during the experimental phase. Consequently, the acquired measurements exhibit localized noise in some intervals, primarily attributed to the mechanical oscillations of the attached object. In Figure 3.14 it is possible to observe the commanded joint velocities to execute the task.



(a)



(b)

Figure 3.12 – Setup for the Visual Servoing task.

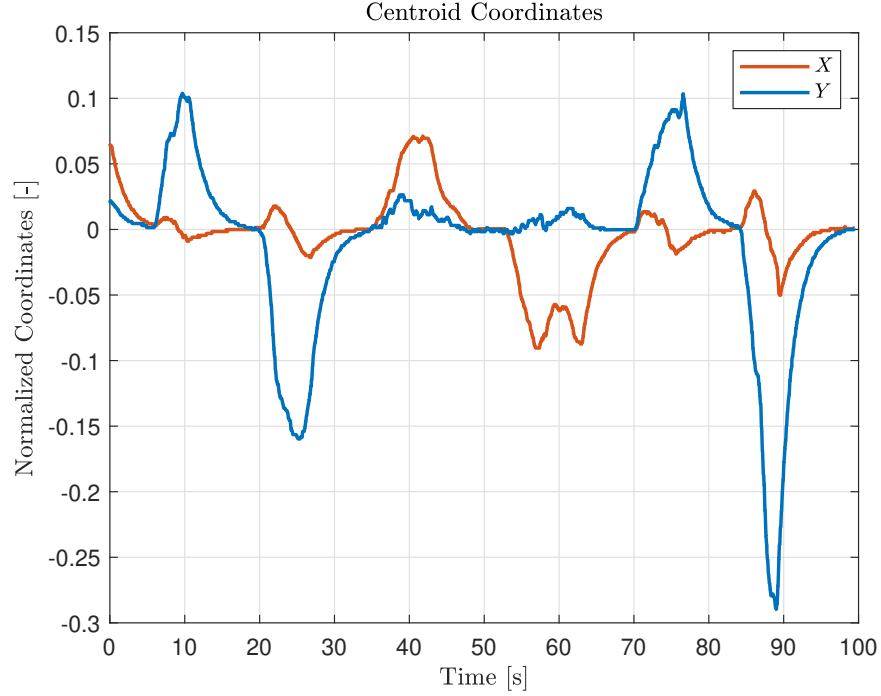


Figure 3.13 – Centroid Coordinates (IBVS).

3.5.2 Visual Servoing Gradient Projected Method

The fact that the dimension of the object centering task is equal to 2, determines the existence of a non-trivial null space of dimension 4, which can be exploited: some joint velocities can be projected into this null space and, without affecting the main task, can perform one or more secondary tasks. In particular, the auxiliary velocities in our case are determined to maximize a functional that takes into account the joint limits, the proximity to singular configurations and, above all, the proximity to the other robot, to avoid collisions. Specifically, the auxiliary velocities are chosen to minimize a multi-objective cost functional, aimed at maximizing safety margins with respect to joint limits, moving the manipulator away from singular configurations and, above all, ensuring collision avoidance with the second robot. The calculation of this last component of the cost functional required a modification of the URDF file, such that the collision volumes based on the original meshes were replaced with geometric primitives (spheres) enclosing the links of the manipulator, as illustrated in Figure 3.15. Such a simplification drastically reduces the computational burden, given that the calculation of the distance between the two manipulators is reduced to the calculation of distances between centers of spheres of known radius.

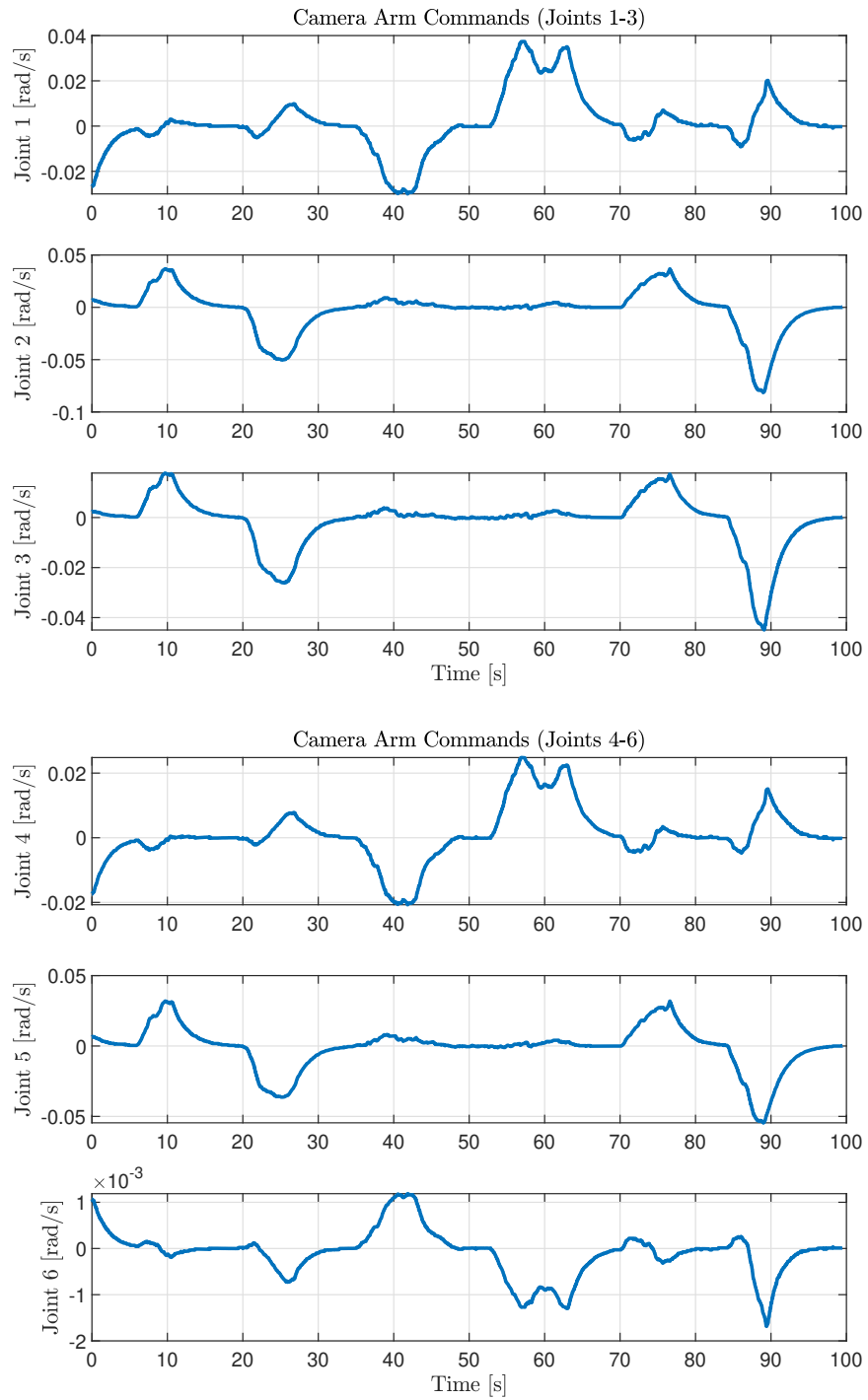


Figure 3.14 – Commanded joint velocities (IBVS).

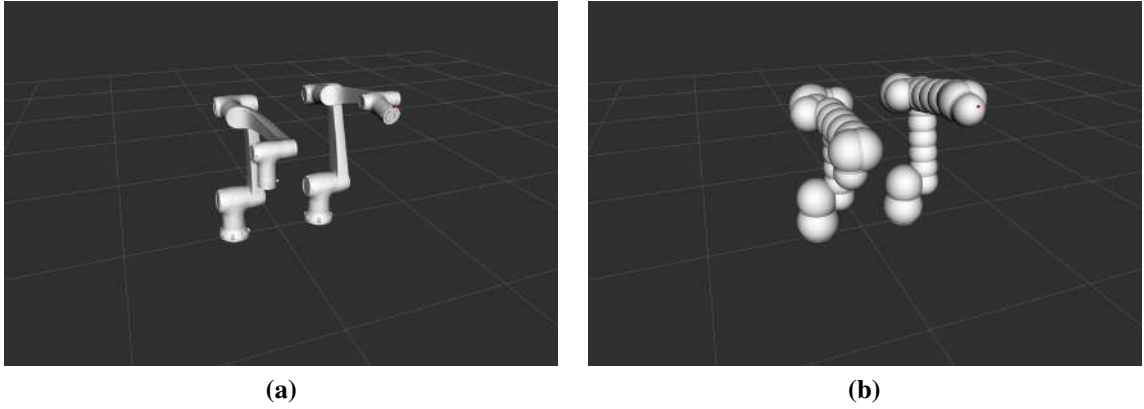


Figure 3.15 – Collisions volumes before (a) and after the change (b).

As mentioned in Section 3.5 and as described in [28] the Pinocchio library allows instantiating both a kinematic model and a `GeometricModel`, which includes the robot's collision volumes (here approximated as spheres), either via the C++ API or by parsing the URDF. Using Pinocchio's forward kinematics functions, the current joint configurations are processed to update the spatial pose of each sphere. From these poses, the Euclidean distances d_{ij} between the spheres of the two manipulators are directly computed and used to evaluate the contribution h_C (see Equation (2.14)) of the cost functional.

To show the capability of the system controlled through this strategy two different scenarios have been considered. In the first case, the secondary manipulator approaches the camera-equipped robot, moving toward its end-effector to test the dynamic collisions avoidance response. In the second scenario while the secondary manipulator executes a grasping operation, the camera-robot must maintain visual tracking of the target object to prevent the loss of visual features, while simultaneously exploiting its functional redundancy to avoid collisions with the gripper-equipped arm. The selected parameter values for the cost function, along with the chosen control law gains, are summarized in Table 3.2. The cost function parameters were tuned through trial and error; more specifically, it was noted that the most significant contributions were those related to joint limits (h_A) and collisions (h_C), given that in the experimental scenario, the robot starts from initial conditions far from kinematic singularities. Furthermore, the contribution related to joint limits was also used to prevent the robot from reaching the workspace limits, which would have been possible

Parameters	ρ_c	α_c	β_c	ρ_J	α_J	β_J	ρ_S	α_S	β_S	k_{r1}	k_{r2}
Value	0.14	8.5	1.0	1.0	1.0	1.0	0.1	0.01	2.0	0.3	0.017

Table 3.2 – PGM Visual Servoing parameters.

if the only contribution had been that related to collisions. This indicates that this contribution is not only relevant for control safety but can also be modified, if necessary, to ensure that the manipulator does not deviate excessively from its initial pose. The adopted cost function and its corresponding contribution to the control action are detailed in Section 2.3 (see, in particular, Equations (2.10)-(2.14)). In both scenarios, the left robot is governed by a standard inverse kinematics algorithm. Consequently, the control law is expressed as

$$\dot{q}_t = J_t^{-1} K_t \tilde{x} \quad (3.2)$$

where the gain matrix is chosen as $K_t = 0.5I_6$. It is worth noting that, to calculate the error in Cartesian space, the $\log_6$ function from the Pinocchio library was used to extract the six components of the error from the homogeneous transformation matrix (for further details on the definition of error used, see 2.1). With regard to the robot equipped with a camera, the corresponding control law is derived by applying the projected gradient approach described in Section 2.3. Its analytical expression is given by

$$\dot{q}_c = k_{r1} (L_s J_c)^\dagger (s_d - s) - k_{r2} (I_6 - (L_s J_c)^\dagger L_s J_c) \nabla H \quad (3.3)$$

3.5.2.1 First Scenario Results (the left robot approached the right one)

The system is composed of two manipulators governed by distinct control logics. The camera-equipped robot is controlled by a visual servoing law based on the minimization of the error defined on the visual feature vector s . The second manipulator, on the other hand, is controlled via an inverse kinematics algorithm, based on an operational space error: the desired pose was chosen such as to invade the workspace of the other manipulator. The experiment is divided into several sequential phases. Initially, the camera-equipped robot performs the tracking task, bringing the visual error to steady-state (Figure 3.16a). Subse-

quently, the secondary robot approaches, triggering the collision avoidance: in Figure 3.16b this repulsive motion can be observed, and in Figure 3.18 this corresponds to the first peak of the cost function. Such local increment is caused both by the decrease of the minimum distance between the collision volumes and by the manipulator approaching its own joint limits (see Figure 3.19). The latter were chosen to prevent an excessive displacement towards the workspace limits: a too pronounced repulsion would indeed entail the object leaving the field of view and the failure of the detection task (recall that the distance from the object is not taken into account by the control law). Subsequently, the second robot returns to its initial pose, allowing the main robot to re-establish the minimum-cost configuration (see Figure 3.16c), as confirmed by the evolution of the cost function. Finally, the second robot approaches again (see also the joint velocities in Figure 3.20), causing another repulsive motion (Figure 3.16d) and a second peak in the cost function. From Figure 3.17 we can observe that the error, after having reached the zero steady-state value, does not increase in the remainder of the experiment. In particular, the settling time is $T_{a2} = 20$ s and the steady-state error is $e_{\infty} = 2 \times 10^{-3}$. Compared to the previous case, a degradation in the settling time can be observed. This is expected, as the control law must now satisfy not only the primary visual servoing task but also the secondary tasks dictated by the minimization of the cost functional. Nevertheless, no differences are observed in the achieved steady-state error, which remains sufficiently low even after subsequent motions executed by the robot to avoid collisions. This confirms that the control achieves the tasks by establishing a hierarchy: the gradients related to collision avoidance and joint limits are projected into the null space of the interaction Jacobian, generating internal motions that do not perturb the execution of the primary task in any way.

3.5.2.2 Second Scenario Results

In this scenario, it was chosen to replicate the object pick task, previously implemented using the MoveIt framework (Section 3.3.2).

The experimental pipeline is divided into four phases: initially, the camera-equipped manipulator frames the target, driving the regulation error in the image space to zero, as shown in Figure 3.22. Subsequently, the second robot performs the grasping maneuver

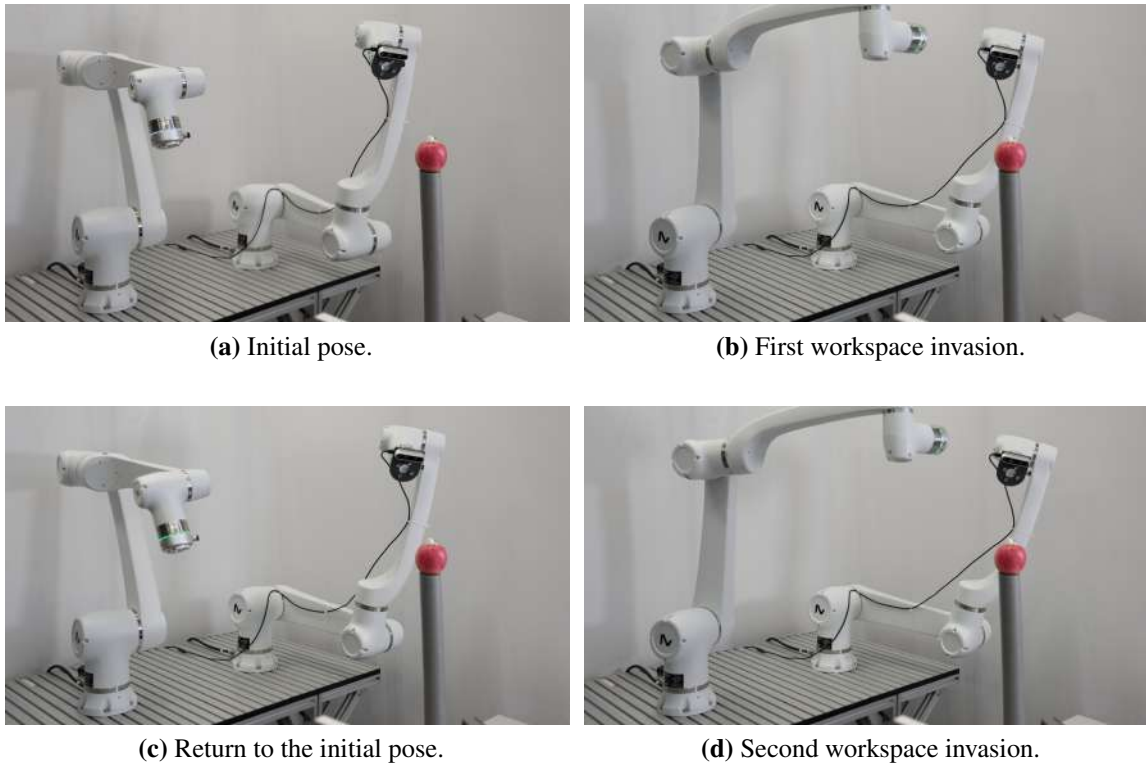


Figure 3.16 – Experiment phases.

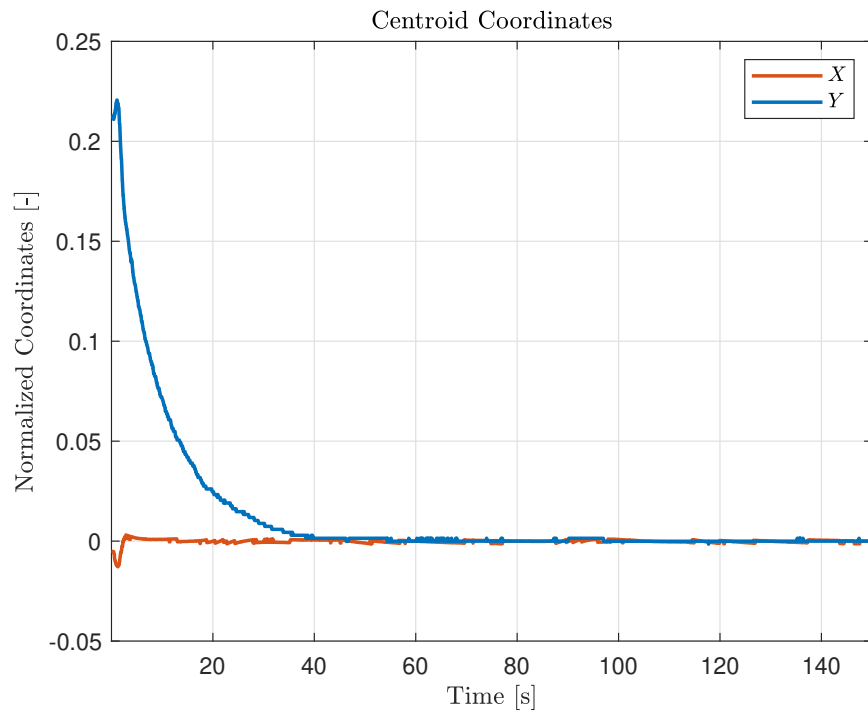


Figure 3.17 – Centroid Coordinates (PGM).

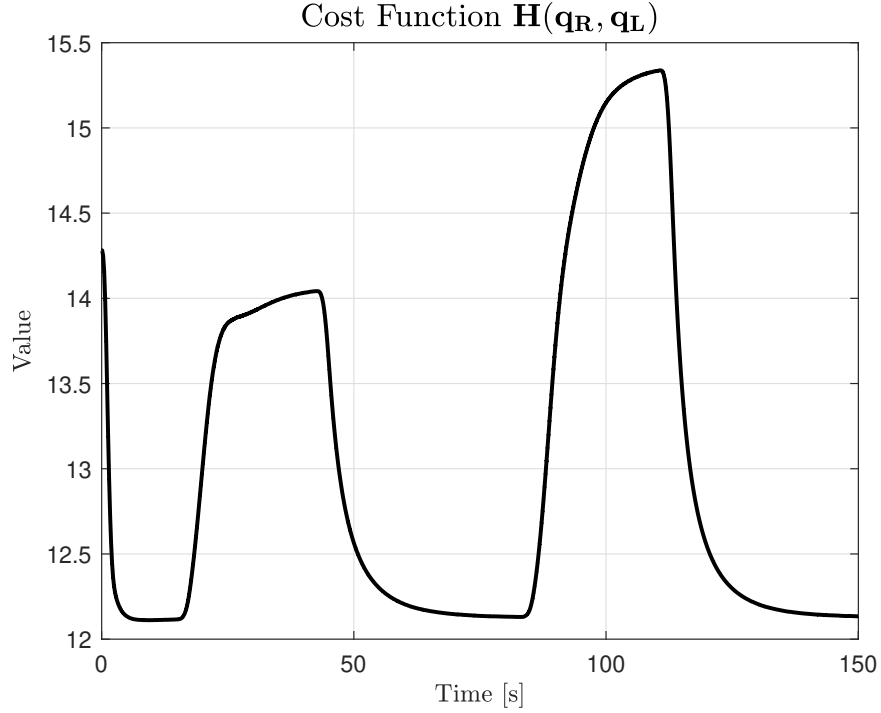


Figure 3.18 – Cost evolution.

and, in response to such intrusion into the operational space, the camera-equipped robot executes a repulsive motion to guarantee collision avoidance (Figures 3.21c-3.21d). The cost function decreases monotonically until it reaches a minimum. Unlike the previous scenario, no localized peaks emerge corresponding to the approach phases of the gripper-equipped robot. This evolution is justified by a less severe interference: the consequent repulsive motion is more contained and does not force the manipulator to approach the joint limits significantly. Numerically, the 2% settling time is $T_{a2} = 23$ s, while the steady-state error is $e_{\infty} = 3 \times 10^{-3}$. The same observations made previously regarding the degradation of the settling time apply to this case as well.

3.5.3 Occlusion-Free Visual Servoing

It is necessary to clarify that the tests, both in simulation and on real hardware, related to the Visual Servoing (VS) with occlusion avoidance task were carried out on a different setup from the original one. Due to time constraints, it was not possible to implement these control schemes on the hardware that was originally used abroad. Specifically, the

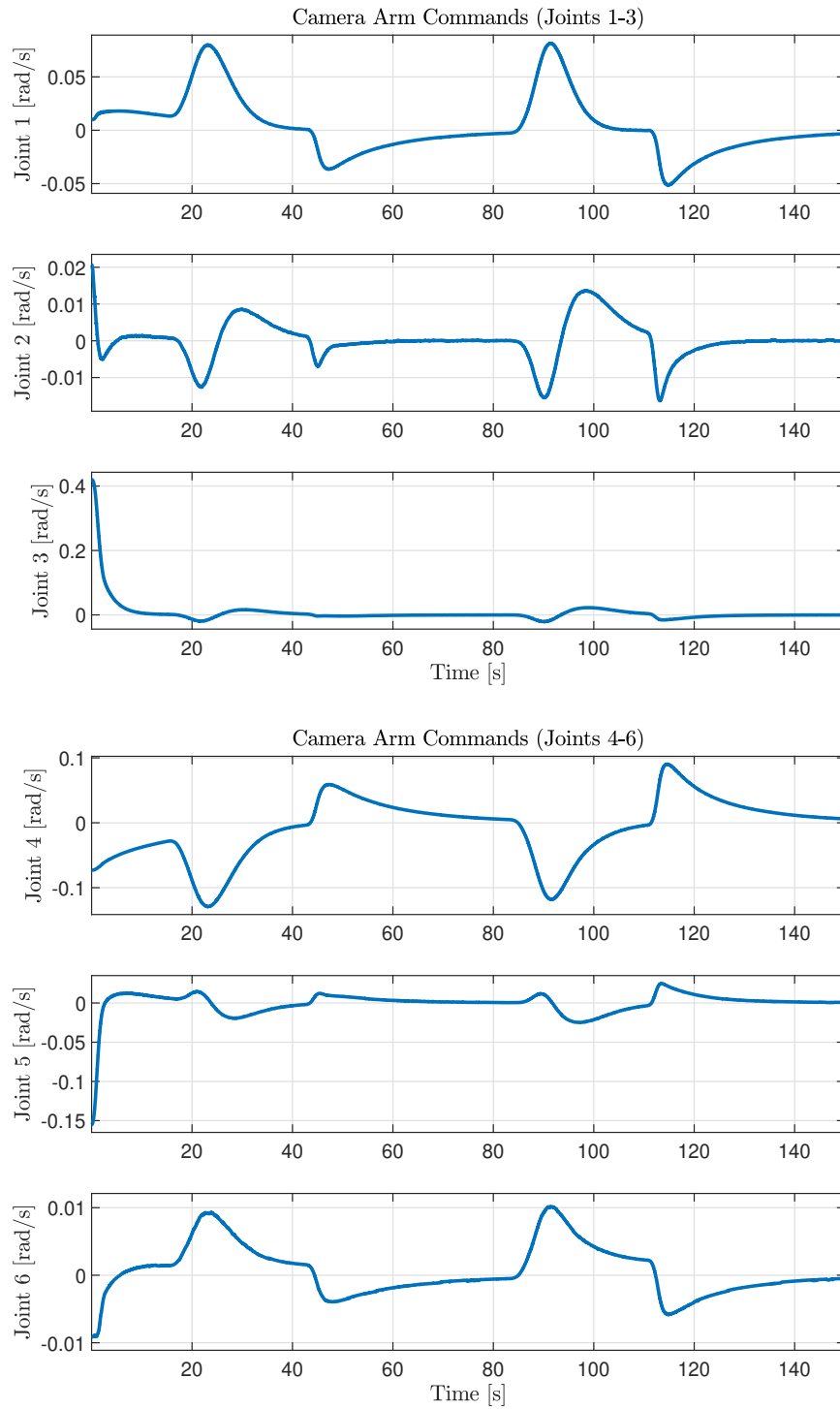
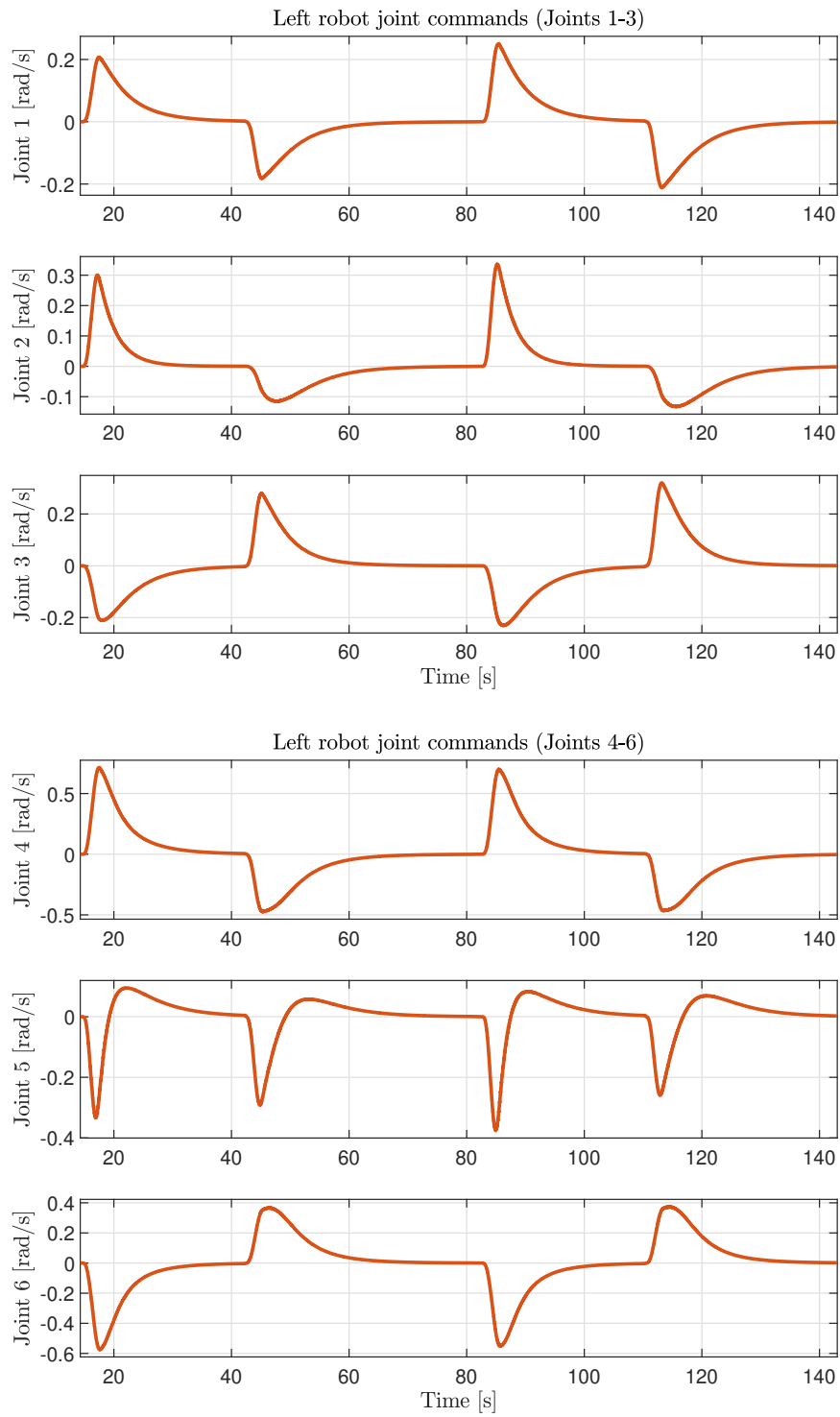


Figure 3.19 – Commanded right joints velocities (PGM).

**Figure 3.20** – Commanded left joints velocities (PGM).

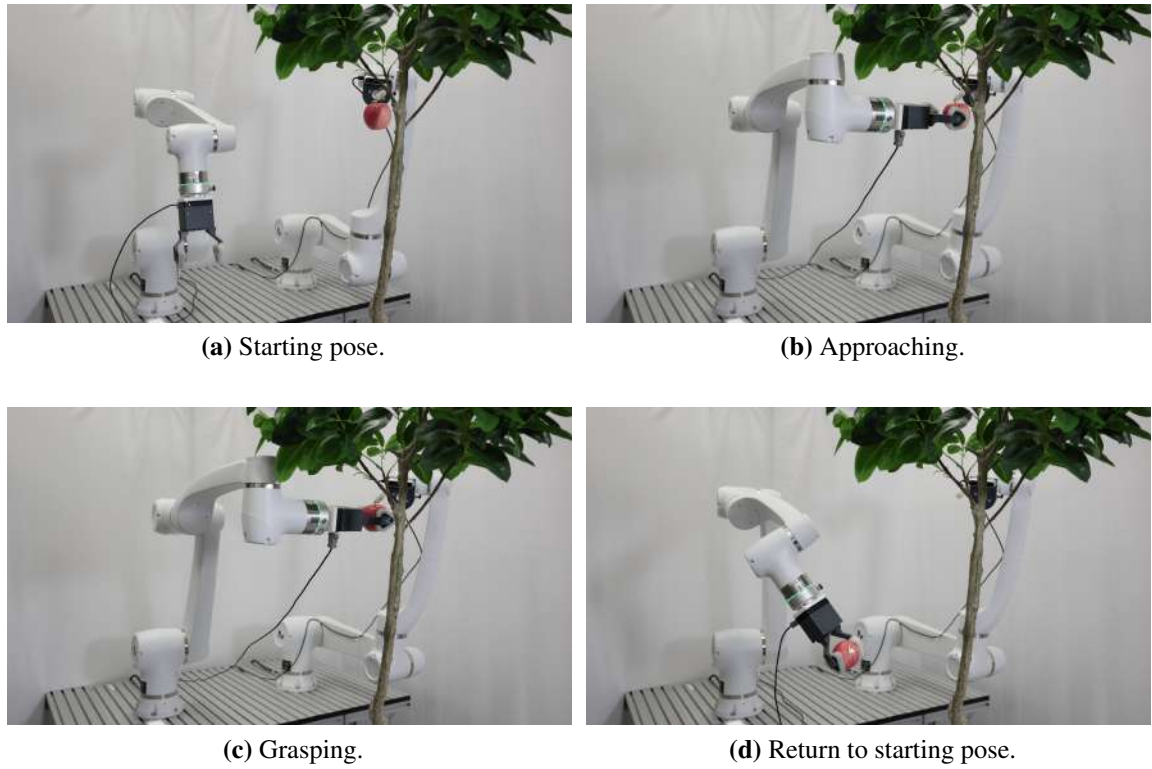


Figure 3.21 – Experiment phases.

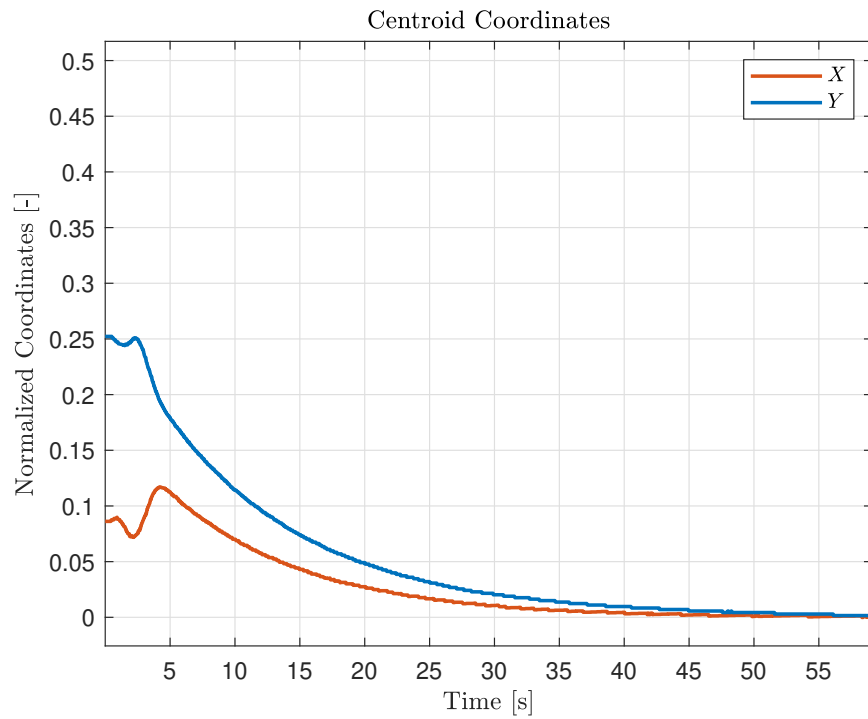


Figure 3.22 – Centroid Coordinates (Grasping task).

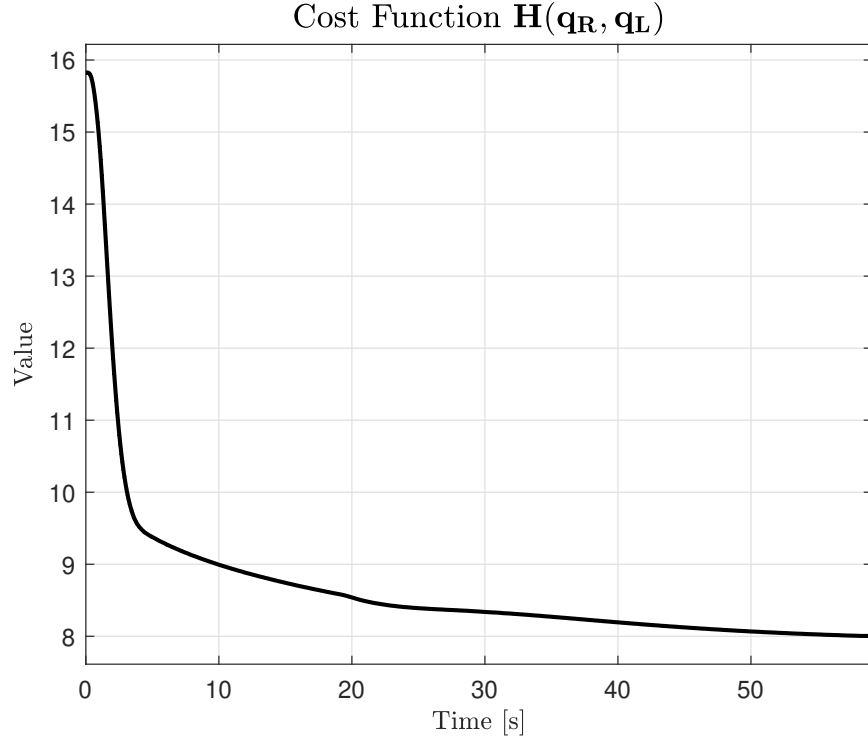


Figure 3.23 – Cost function (Grasping task).

new experimental setup consists of two 7-degree-of-freedom (DoF) Franka Emika Panda robots as depicted in Figure 3.4¹. To perform obstacle detection, the OpenCV [30] library was used. More specifically, it was employed to carry out color-based image segmentation, and once the boundaries of the objects of interest were identified, the bounding boxes were defined. To determine the depth of the bounding boxes, a simple yet conservative approach was adopted: specifically, the pixel at the lower depth was considered; in this way, it is as if the object whose occlusion should be avoided were approximated by a virtual screen, and consequently the constraints are more approximate but also more conservative than they would be if the actual geometry of the object were taken into account. The experiment was structured so that the robot equipped with a camera has, as its primary task, that of tracking the end-effector of the other robot, so that it always remains within the camera's FoV; on the other hand, the desired joint velocities for the other robot are sent *via* joystick, and an obstacle that can occlude the camera's view has been placed in the scene (Figure 3.24). Consequently, the velocity of the robot equipped with the camera was chosen as in Equa-

¹The ROS 2 packages of the occlusion-free visual servoing on the Franka Emika setup are available at: https://github.com/salvatoredeluca/franka_sasi

tion (2.10): an action in the null space of the primary task was also included, not so much to meet collision-avoidance requirements, but rather to prevent the manipulator from moving into singular configurations or reaching joint limits. The parameters adopted for the projected gradient method in this specific setup are summarized in Table 3.4. Compared to the previous experiments (Table 3.2), the gains and weights associated with the secondary cost functions are generally lower. This choice is motivated by the fact that the considered setup is not expected to drive the manipulators close to singular configurations or joint limits. Nevertheless, this null-space action was included to fully exploit the available functional redundancy and to provide an additional safety margin during motion execution. Moreover, in this case, since the point of interest coincides with the end-effector of the other manipulator, the image-coordinate vector s was obtained by exploiting the kinematics of the latter; specifically, the pose of the end-effector was expressed with respect to the camera reference frame and subsequently projected onto the image plane. As for the velocity of the robot on the right, it is controlled manually via joystick; the physical signals are acquired using the ROS 2 joy package. Ultimately, based solely on the visual information from a single camera, both robots will deviate more or less significantly from their respective reference velocities in order to prevent the target from being occluded. The nodes and topics architecture to perform such a task is depicted in Figure 3.25: in particular a node `/edges_detection_node` detects the edges of the bounding box of the occluding boxes and publishes them to a topic `/edges` (a custom message interface was created in order to send the list of the points in the space defining the bounding boxes); then, finally, the `/occlusion_task_node` subscribed to the `/edges`, `/tf` and the `/joint_states` of both the manipulators, once solved the QP constrained optimization problem (see Equation (2.21) in Section 2.4 for details), sends the optimal velocities to the controllers. To solve the QP problem, the Python library `qpsoolvers` was used, and to evaluate its performance the computation time at each cycle was analyzed, determining the mean, median, standard deviation, and maximum of the samples obtained during the experiment.



Figure 3.24 – Detailed view of the left manipulator with the occluding object.

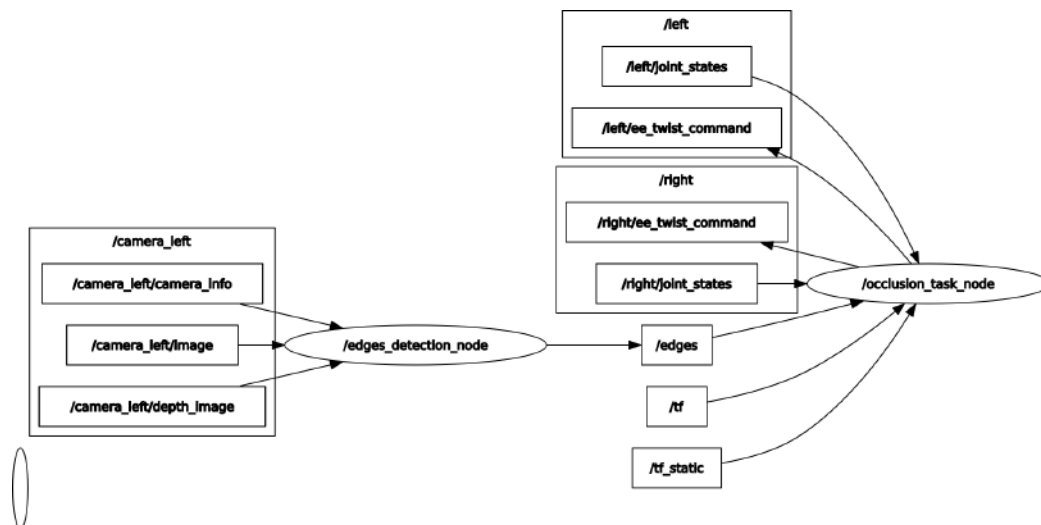


Figure 3.25 – Nodes and topics architecture for the Occlusion-Free IBVS.

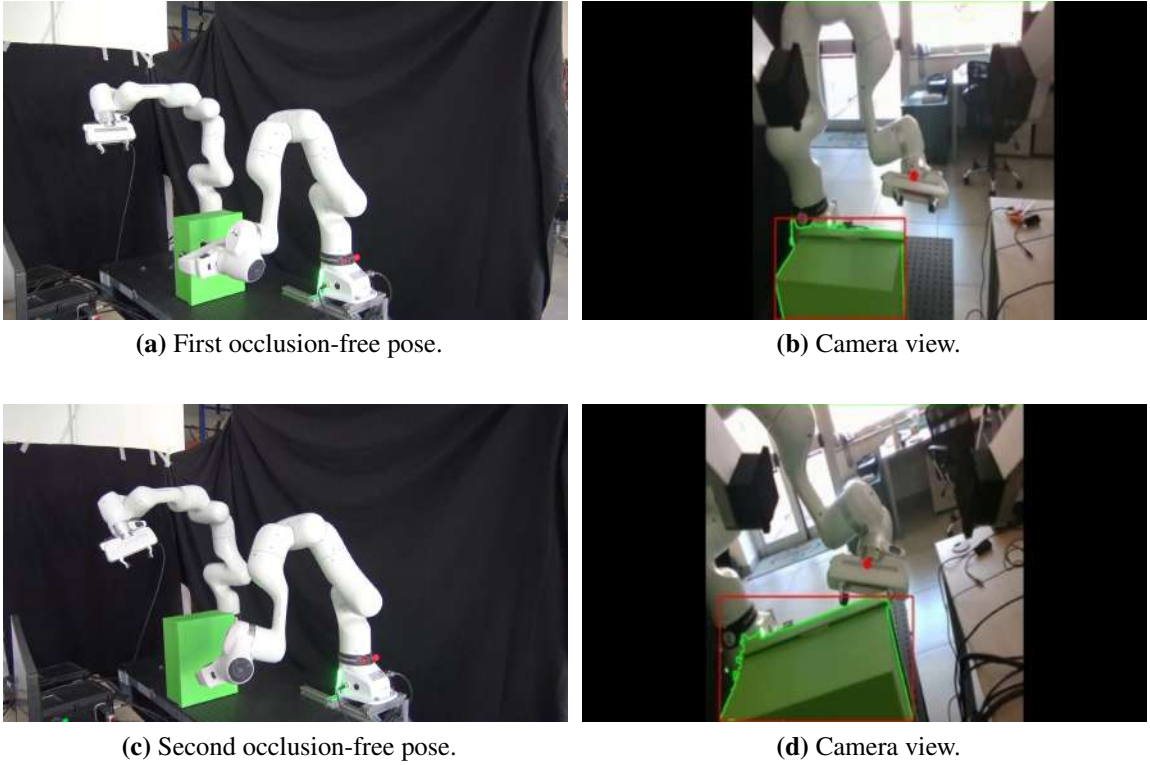


Figure 3.26 – Occlusion-free visual servoing results.

3.5.3.1 Results

It should be specified that, unlike in the previous cases, the joint velocities were not plotted, but rather the commanded twists. This change in approach is due to the modification of the setup of the two manipulators, whose interface drivers receive commands in terms of twists instead of joint velocities. Figure 3.26 depicts two phases of the experiment in which the manipulator to be observed is potentially obstructed by the occluding object (Figures 3.26a-3.26c). From the camera images (Figures 3.26b-3.26d), it can be seen that the point of interest (indicated by a red dot) remains continuously visible and does not intersect the obstacle, which, for the purposes of the algorithm, is represented by its bounding box as explained above. Furthermore, Figure 3.27, which shows the evolution of the error in the image space, confirms that the visual servoing task is successfully accomplished and the point of interest on the arm is consistently maintained within the camera's field of view. From Figures 3.28-3.29, it can be observed that the commanded velocities are not perfectly smooth as a consequence of using a QP solver to control the robots. Table 3.3

Statistical Metric	Execution Time (ms)
Worst-Case Execution Time (WCET)	4.96
Mean	2.01
Median	1.98
Standard Deviation (σ)	0.42

Table 3.3 – Analysis of the QP solver execution time per cycle.

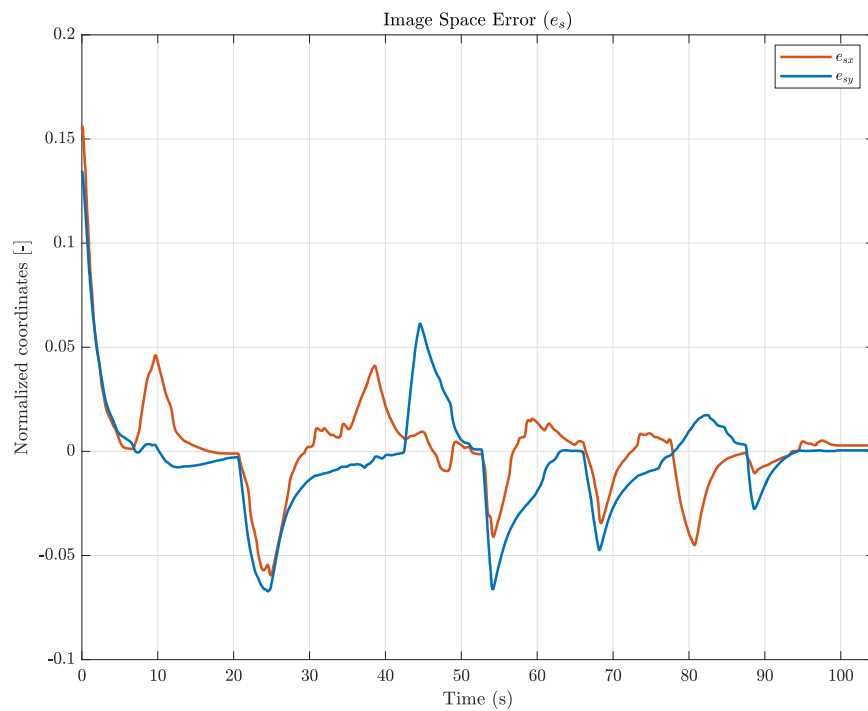


Figure 3.27 – Image Space Error Occlusion-free VS.

summarizes the statistical parameters relevant for evaluating the solver’s performance. In particular, the Worst-Case Execution Time (WCET) is significantly lower than the period of the `/occlusion_task_node`, highlighting the strong temporal robustness of the architecture; given a cycle period of about 33 ms, solving the QP problem in fact occupies only 6.1% of the available time. The coefficient of variation, equal to 0.21, confirms the low variability of the computation time, while the close proximity between the mean and median values indicates a concentration of the measurements around 1.98 ms: the system is therefore sufficiently deterministic.

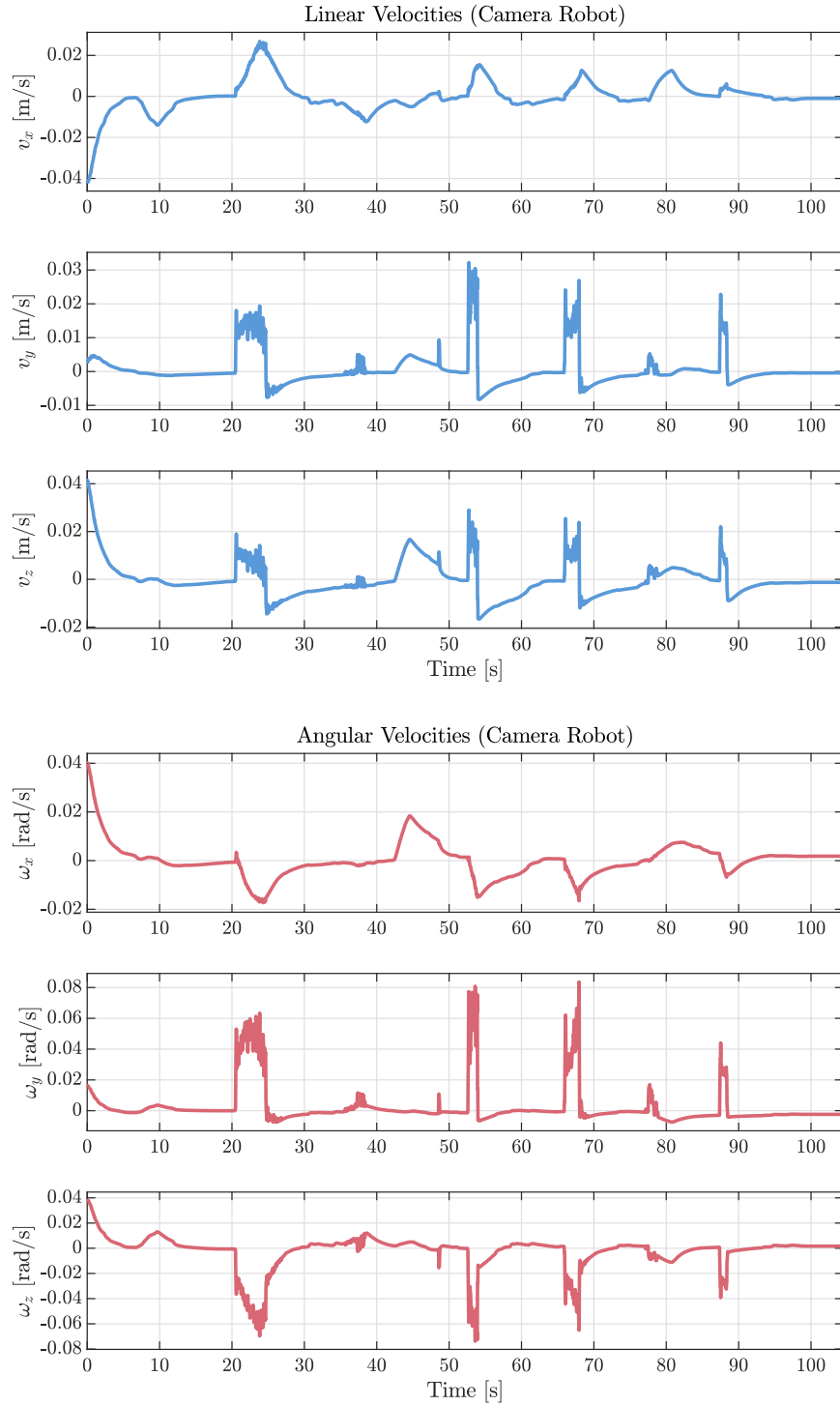
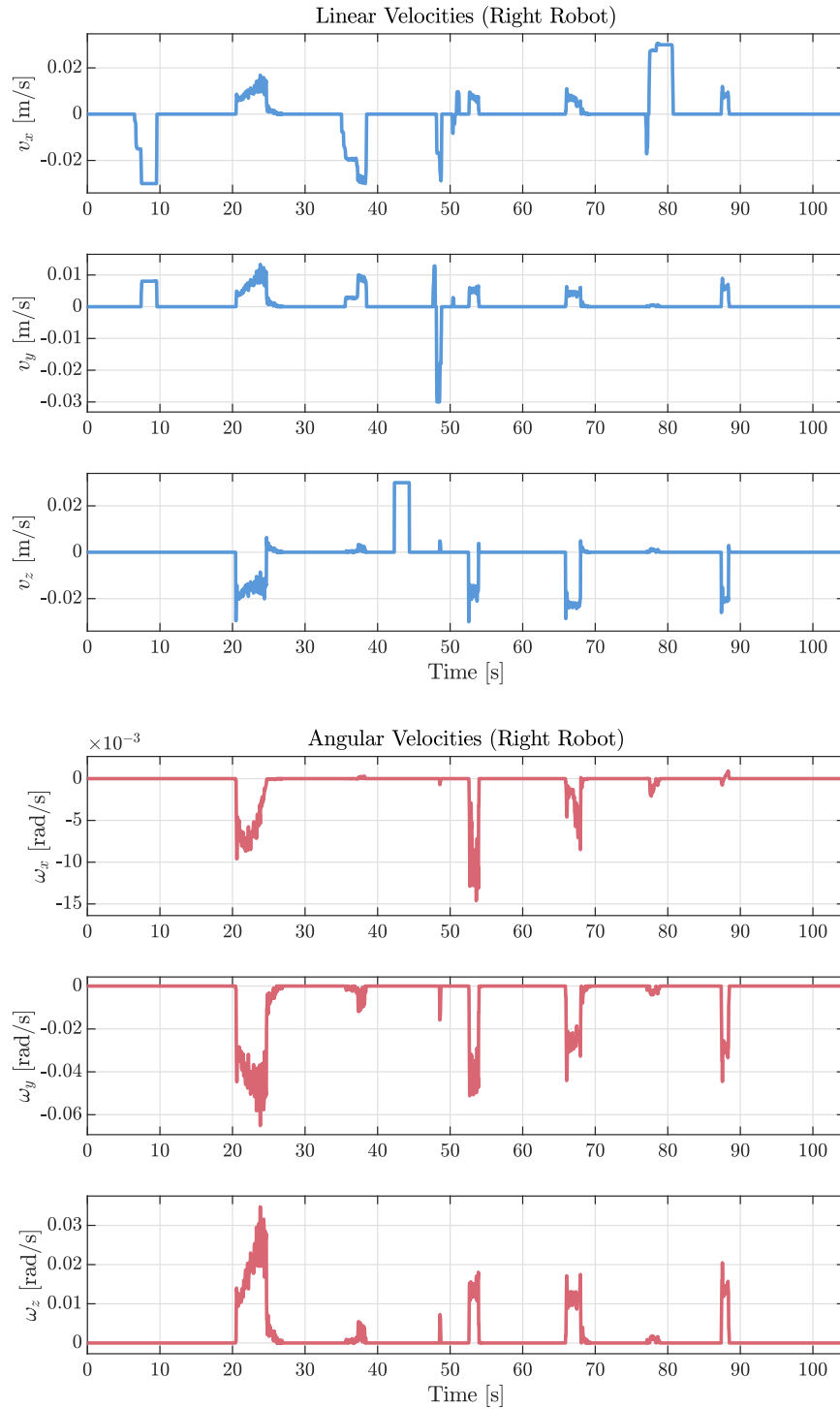


Figure 3.28 – Commanded Twist (Camera Arm).

Parameters	ρ_c	α_c	β_c	ρ_J	α_J	β_J	ρ_S	α_S	β_S	k_{r1}	k_{r2}
Value	0.004	1.0	1.0	0.058	10.0	1.0	0.0007	10	1.0	0.46	0.03

Table 3.4 – PGM Visual Servoing parameters in the Occlusion-free VS Case.

**Figure 3.29** – Commanded Twist (Right Arm).

Chapter 4

Conclusions

This thesis presented the design, implementation, and experimental validation of a control architecture for a dual-arm robotic system. A first control package, based on the MoveIt framework, was developed and experimentally shown to enable synchronous motion of the two manipulators while avoiding mutual collisions. Building on this result, a second architecture was proposed, adopting a task-priority strategy that combines a closed-loop inverse kinematics algorithm for one manipulator with the projected gradient method for the camera-equipped one, allowing the latter to react to the motion of the former and ensuring the safe execution of collaborative tasks. A fundamental and necessary prerequisite for the implementation and validation of these control systems was the development, from scratch, of an open-source ROS 2 interface for the two newly acquired Neura LARA 8 manipulators. This interface decoupled their control from the manufacturers' proprietary software and enabled the synchronous, independent management of both arms through a custom control architecture. This effort allowed all the proposed control schemes to be successfully validated experimentally on the real platform, and it also provided the hosting laboratory with a reusable, open-source framework for future research on the acquired system. In parallel, the work carried out at the robotics laboratory of the University of Naples Federico II made it possible to address and validate an occlusion-avoidance strategy on a Franka Emika dual-arm experimental setup, with the corresponding control scheme also being successfully validated experimentally. Overall, solving the technical challenges related to the low-level interfacing of the Neura LARA 8 manipulators turned two previously

isolated platforms into a functioning multi-robot system ready for advanced research, providing a reusable infrastructure for future projects at the laboratory. Like any experimental work, this thesis also presents some limitations, mainly due to the trade-off between infrastructure development and the development of advanced control schemes. The need to devote a substantial part of the project to solving technical issues and setting up the ROS 2 architecture inevitably limited the range of control techniques that could be tested on the LARA 8 platform within the available time. Moreover, the occlusion-avoidance algorithm was validated on a separate experimental setup, as it was not possible, given the time available, to integrate it directly on the Neura platform. Future developments of this work could proceed along two main methodological directions. On the one hand, the reliability of the occluding-obstacle detection could be improved by integrating more robust object detection techniques based on neural networks, such as YOLOv8, in place of the current color-based detection approach implemented with OpenCV. On the other hand, the quadratic-programming-based occlusion-avoidance strategy could be extended to explicitly address the discontinuities that may arise in the commanded joint velocities when the occlusion constraint is activated or deactivated. A possible direction in this sense is the adoption of a Model Predictive Control formulation, which would allow the future evolution of the constraint to be anticipated over a prediction horizon rather than reacted to instantaneously, or, alternatively, the introduction of additional smoothness constraints directly within the QP solver, penalizing abrupt variations in the commanded velocities between control cycles.

Bibliography

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>
- [2] Y. Jo, Y. Park, J. Seol, J. Pak, B. Kim, C. Kim, and H. I. Son, “A review on dual-arm manipulation in agriculture,” *IEEE Access*, vol. 13, pp. 150 379–150 399, 2025.
- [3] S. Stavridis, L. Droukas, and Z. Doulgeri, “Bimanual grape manipulation for human-inspired robotic harvesting,” *IEEE/ASME Transactions on Mechatronics*, vol. 30, no. 4, pp. 2722–2732, 2025.
- [4] D. Rodríguez-Nieto, E. Navas, and R. Fernández, “Automated seeding in hydroponic greenhouse with a dual-arm robotic system,” *IEEE Access*, vol. 13, pp. 30 745–30 761, 2025.
- [5] Y. Jiang, S. Zhao, J. Liu, S. Wu, Y. Jiang, and Y. Jin, “Review of dual-arm parallel and collaborative motion: methods, progress and applications in agriculture,” *Computers and Electronics in Agriculture*, vol. 239, p. 111081, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0168169925011871>
- [6] J. Ahn, C. Kim, and C. Nam, “Coordination of two robotic manipulators for object retrieval in clutter,” in *2022 International Conference on Robotics and Automation (ICRA)*, Philadelphia, PA, USA, 2022, pp. 1039–1045.

- [7] E. Gursoy, B. Navarro, A. Cosgun, D. Kulić, and A. Cherubini, “Towards vision-based dual arm robotic fruit harvesting,” in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, Auckland, New Zealand, 2023, pp. 1–6.
- [8] M. Selvaggio, F. Abi-Farraj, C. Pacchierotti, P. Robuffo Giordano, and B. Siciliano, “Haptic-based shared-control methods for a dual-arm system,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4249–4256, 2018.
- [9] D. Papageorgiou, L. Koutras, and Z. Doulgeri, “A controller for reaching and unveiling a partially occluded object of interest with an eye-in-hand robot,” in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, Ginowan, Okinawa, Japan, 2022, pp. 254–260.
- [10] D. Nicolis, M. Palumbo, A. M. Zanchettin, and P. Rocco, “Occlusion-free visual servoing for the shared autonomy teleoperation of dual-arm robots,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 796–803, 2018.
- [11] Y. Wang, T. Ye, and X. Xu, “Hierarchical control for occlusion-free visual servoing of robotic manipulators,” *IEEE Transactions on Control Systems Technology*, vol. 34, no. 4, pp. 1911–1922, 2026.
- [12] S. Wei, B. Dai, R. Khorrambakht, P. Krishnamurthy, and F. Khorrami, “Diffocclusion: Differentiable optimization based control barrier functions for occlusion-free visual servoing,” *IEEE Robotics and Automation Letters*, vol. 9, no. 4, pp. 3235–3242, 2024.
- [13] J. Shao, Y. Gan, and X. Dai, “Autonomous path planning and realization for dual robot cooperation based on ros framework,” in *2023 International Conference on Advanced Robotics and Mechatronics (ICARM)*, Sanya, China, 2023, pp. 1065–1070.
- [14] I. A. Şucan, M. Moll, and L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, <https://ompl.kavrakilab.org>.

- [15] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, UK: Cambridge University Press, 2017.
- [16] D. Sepúlveda, R. Fernández, E. Navas, M. Armada, and P. González-De-Santos, “Robotic aubergine harvesting using dual-arm manipulation,” *IEEE Access*, vol. 8, pp. 121 889–121 904, 2020.
- [17] M. Görner, R. Haschke, H. Ritter, and J. Zhang, “Moveit! task constructor for task-level motion planning,” in *2019 International Conference on Robotics and Automation (ICRA)*, Montreal, QC, Canada, 2019, pp. 190–196.
- [18] A. Bonci, F. Gaudeni, M. C. Giannini, and S. Longhi, “Robot operating system 2 (ros2)-based frameworks for increasing robot autonomy: A survey,” *Applied Sciences*, vol. 13, no. 23, 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/23/12796>
- [19] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London: Springer, 2009.
- [20] B. Siciliano and J.-J. E. Slotine, “A general framework for managing multiple tasks in highly redundant robotic systems,” in *Fifth International Conference on Advanced Robotics 'Robots in Unstructured Environments'*, vol. 2. IEEE, 1991, pp. 1211–1216.
- [21] A. M. Zanchettin and P. Rocco, “Path-consistent safety in mixed human-robot collaborative manufacturing environments,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Tokyo, Japan, 2013, pp. 1131–1136.
- [22] Neura Robotics GmbH, *LARA User Manual*, Neura Robotics GmbH, Metzingen, Germany, October 2024, version 3.2.1, Document number: 100002000001.
- [23] —, *ROS Documentation*, Neura Robotics GmbH, Metzingen, Germany, 2025, version 1.0, Document number: 0100001000005. [Online]. Available: <https://www.neura-robotics.com>

-
- [24] ———, *Neura CPP API*, Neura Robotics GmbH, May 2025, release 1.1.
- [25] ———, *Neurapy*, Neura Robotics GmbH, May 2025, release v4.22.4.
- [26] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3. IEEE, 2004, pp. 2149–2154.
- [27] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” 2023, version 8.0.0, Computer software. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [28] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiriaux, O. Stasse, and N. Mansard, “The Pinocchio C++ library – A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives,” in *International Symposium on System Integration (SII)*, 2019.
- [29] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi – A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [30] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.

Web References

- [31] “ROS2 Jazzy Jalisco Documentation,” Open Robotics, 2024, accessed: 19 November 2025. [Online]. Available: <https://docs.ros.org/en/jazzy/index.html>
- [32] “ROS2 Control Documentation - Jazzy,” ros-controls contributors, 2024, accessed: 10 January 2026. [Online]. Available: <https://control.ros.org/jazzy/index.html>
- [33] “Moveit 2 documentation,” PickNik Robotics, 2024, accessed: 15 November 2025. [Online]. Available: <https://moveit.picknik.ai/main/index.html>
- [34] “Roboflow computer vision platform,” Roboflow Inc., 2024, accessed: 25 November 2025. [Online]. Available: <https://roboflow.com/>
- [35] Open Source Robotics Foundation, Inc. (2026) message_filters — ros 2 message filters documentation. Online. Accessed: 2026-02-26. [Online]. Available: https://docs.ros.org/en/jazzy/p/message_filters/
- [36] PickNik Robotics, “MoveIt Servo Documentation - Realtime Arm Control,” https://moveit.picknik.ai/main/doc/examples/realtime_servo/realtime_servo_tutorial.html, 2024.